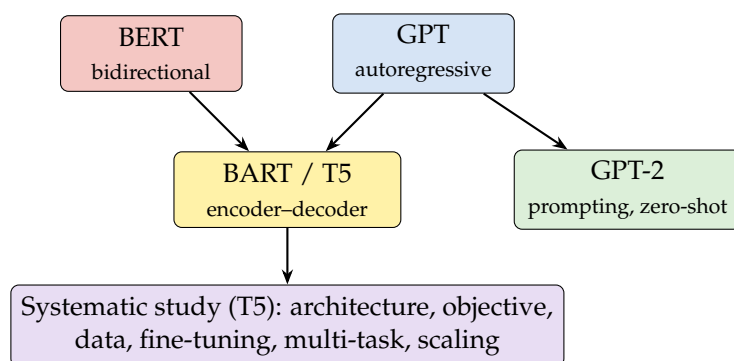


Introduction to Large Language Models

Week 7 — Detailed Notes

Encoder–Decoder models, BART, GPT-2, T5,
and the choices that affect performance



Based on the lectures of Prof. Mitesh M. Khapra (AI4Bharat, IIT Madras)

Notes prepared for Deon Dmello — IITM BS, LLM course

Contents

0	Prerequisites: a compact recap (not in slides)	4
0.1	Notation	4
0.2	Scaled dot-product attention with shapes	5
0.3	The three attention flavours	5
0.4	The two pre-training objectives we already know	6
1	From BERT to BART, and GPT-2	7
1.1	Two approaches, two strengths	7
1.2	BART: Bidirectional AutoRegressive Transformer	7
1.2.1	Architecture	7
1.2.2	GELU activation	8
1.2.3	The noising transformations	9
1.2.4	Training objective	9
1.3	GPT-2: Prompting large language models	10
1.3.1	The problem with “pre-train, then fine-tune”	10
1.3.2	Zero-shot transfer by changing the input	11
1.3.3	Scale: model and data	11
1.3.4	Zero-shot performance across domains	12
1.3.5	A promising direction	14
2	Choices that affect performance	15
2.1	Why does one model beat another?	15
2.2	The design space	16
2.3	Wishlist	16
2.4	A systematic study: T5	16
2.4.1	The key idea: everything is text-to-text	17
2.4.2	Dataset: the Colossal Clean Crawled Corpus (C4)	18
3	Pre-training and fine-tuning a baseline model	19
3.1	The baseline architecture	19

3.2	The denoising objective: span corruption	19
3.3	Pre-training setup	20
3.3.1	Optimizer: AdaFactor	21
3.3.2	Learning-rate schedule: inverse square root	21
3.4	Fine-tuning setup	22
3.5	Baseline performance	22
3.5.1	Is this comparable to BERT?	23
4	Experimenting with different architectures	24
	The three architectural variants	24
	Mask patterns	24
	Prefix language modelling	25
	Comparing architectures fairly: parameters vs. FLOPs	25
	Results: architecture and objective	26
4.1	Module 4.1: Effect of unsupervised objectives	28
4.1.1	Example (input, target) pairs for each objective	28
4.1.2	High-level objectives	28
4.1.3	Corruption strategies	29
4.1.4	Corruption rate	29
4.1.5	Corrupted span length	30
4.2	Module 4.2: Effect of the size of the pre-training dataset	31
4.2.1	Quality vs. quantity	31
4.2.2	Does repeating data hurt?	32
5	Fine-tuning strategies	34
5.1	The default: fine-tune all parameters	34
5.2	Adapter layers	34
5.2.1	Results	35
5.3	Gradual unfreezing	36
6	Multi-task learning	38
6.1	Motivation	38
6.2	General multi-task learning	38
6.3	How to mix: sampling strategies	39
6.3.1	Examples-proportional mixing	39
6.3.2	Temperature-scaled mixing	39
6.3.3	Results	40
6.4	Multi-task learning followed by fine-tuning	41
6.5	Three variants	42

6.5.1 Results	42
7 What if you have more compute budget?	44
7.1 Four ways to spend $4\times$ compute	44
7.2 Results	45
7.3 Parameters or training steps?	45
7.4 Pushing the limits: the final T5 models	46
7.5 Final performance	47
References	48
A One-page revision sheet	49
A.1 Formula sheet	50
A.2 Numbers worth memorising	50
B Numerical problems with worked solutions	51
C Subjective questions with model answers	58
D Multiple-choice questions with answers and explanations	65
D.1 Module 1: BART and GPT-2	65
D.2 Module 2: choices and T5	67
D.3 Module 3: baseline	67
D.4 Module 4: architectures	69
D.5 Module 4.1: objectives	70
D.6 Module 4.2: data	70
D.7 Module 5: fine-tuning	71
D.8 Module 6: multi-task learning	72
D.9 Module 7: scaling	73
D.10 Answer key	74

Module 0

Prerequisites: a compact recap (not in slides)

Week 7 builds on everything we did with BERT and GPT. Before we combine them into encoder-decoder models, it pays to have a precise, dimension-annotated picture of the building blocks. This chapter is a refresher; if the notation is comfortable, skip to Module 1.

How to read these notes

- Each chapter corresponds to one lecture module (Module 1 ... Module 7, including 4.1 and 4.2).
- **Blue boxes** give intuition, **green boxes** give the takeaway the lecture wants you to remember, **grey boxes** contain derivations, **purple boxes** go slightly beyond the slides (useful context, flagged so you know it is extra), and **yellow boxes** point out small inconsistencies in the slides so they do not confuse you in an exam.
- Shapes are written in **red brackets**, e.g. $X [T \times d_{\text{model}}]$.
- The last three chapters are for self-testing: numerical problems, subjective questions and MCQs, all with worked answers.

0.1 Notation

Symbol	Meaning
T	sequence (context) length in tokens
B	batch size (number of sequences per step)
V	vocabulary size
d_{model}	model (hidden) dimension
d_{ff}	inner dimension of the feed-forward network (usually $4d_{\text{model}}$)
h	number of attention heads; $d_k = d_v = d_{kv}$ = per-head dimension
L	number of layers (blocks) in a stack
θ	all learnable parameters
P	number of parameters (used in the T5 comparison)
M	FLOPs needed to process one (input, target) pair

0.2 Scaled dot-product attention with shapes

Given a sequence of T token representations $X [T \times d_{\text{model}}]$, one head computes

$$Q = XW_Q [T \times d_k], \quad K = XW_K [T \times d_k], \quad V = XW_V [T \times d_v], \quad (1)$$

with $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. Then

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\underbrace{\frac{QK^\top}{\sqrt{d_k}}}_{[T \times T]} + \underbrace{\mathcal{M}}_{\text{mask } [T \times T]} \right) \underbrace{V}_{[T \times d_v]} [T \times d_v]. \quad (2)$$

The mask $\mathcal{M}$ holds 0 where attention is allowed and $-\infty$ where it is not; after the softmax, $-\infty$ becomes a weight of exactly 0. **Masks are the single most important idea in this week:** BERT, GPT, prefix-LM and encoder-decoder models differ mainly in which cells of this $T \times T$ matrix are allowed.

Derivation : why divide by $\sqrt{d_k}$

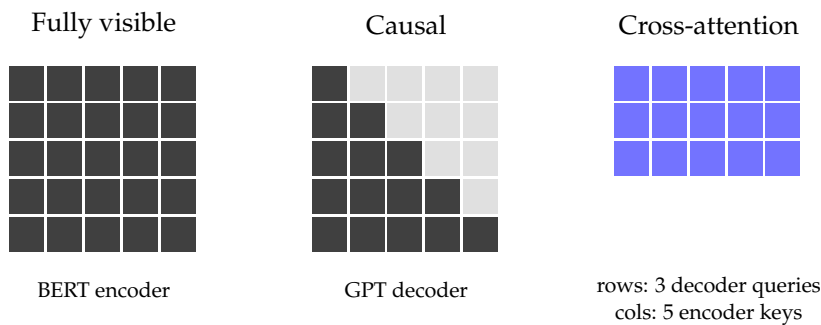
If the entries of q and k are independent with mean 0 and variance 1, then $q^\top k = \sum_{i=1}^{d_k} q_i k_i$ has mean 0 and variance $\sum_i \text{Var}(q_i k_i) = d_k$. So the raw scores have standard deviation $\sqrt{d_k}$. Large scores push the softmax into saturation (one weight ≈ 1 , gradients ≈ 0). Dividing by $\sqrt{d_k}$ brings the variance back to 1.

Multi-head attention concatenates h heads and projects back:

$$\text{MHA}(X) = \underbrace{\text{Concat}(\text{head}_1, \dots, \text{head}_h)}_{[T \times hd_v]} \underbrace{W_O}_{[hd_v \times d_{\text{model}}]} [T \times d_{\text{model}}]. \quad (3)$$

Usually $hd_k = d_{\text{model}}$, so an attention sub-layer has $4d_{\text{model}}^2$ weights (W_Q, W_K, W_V, W_O), ignoring biases.

0.3 The three attention flavours



Rows are queries (the token doing the looking), columns are keys (the token being looked at). Dark = allowed.

- **Self-attention, fully visible (encoder):** every token sees every token. Good for *understanding*.
- **Self-attention, causal (decoder):** token i sees only tokens $\leq i$. Needed for *generation*, because at inference the future does not exist yet.

- **Cross-attention (decoder $\rightarrow$ encoder):** queries come from the decoder, keys and values from the final encoder output $H^{\text{enc}} [T_{\text{in}} \times d_{\text{model}}]$. Shapes: $Q [T_{\text{out}} \times d_k]$, $K, V [T_{\text{in}} \times d_k]$, score matrix $[T_{\text{out}} \times T_{\text{in}}]$. No mask is needed, since the whole input is known.

0.4 The two pre-training objectives we already know

Causal language modelling (CLM, GPT). For $x_{1:T}$,

$$\mathcal{L}_{\text{CLM}}(\theta) = - \sum_{i=1}^T \log P_{\theta}(x_i \mid x_1, \dots, x_{i-1}). \quad (4)$$

Every position produces a training signal (T predictions per sequence).

Masked language modelling (MLM, BERT). Choose a set $\mathcal{M}$ of $\approx 15\%$ positions and corrupt them (80% [MASK], 10% random token, 10% unchanged):

$$\mathcal{L}_{\text{MLM}}(\theta) = - \sum_{i \in \mathcal{M}} \log P_{\theta}(x_i \mid \tilde{x}), \quad (5)$$

where $\tilde{x}$ is the corrupted sequence. Only $\approx 15\%$ of positions produce a signal, but each prediction sees context on *both* sides.

Intuition

CLM $\Rightarrow$ “predict the next word” $\Rightarrow$ natural for generation.

MLM $\Rightarrow$ “fill in the blank using both sides” $\Rightarrow$ natural for comprehension (classification, extractive QA).

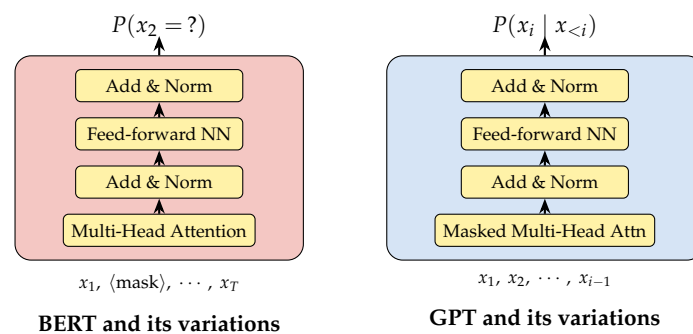
This week asks one question: *can we get the best of both, and which of the many design choices actually matters?*

Module 1

From BERT to BART, and GPT-2

1.1 Two approaches, two strengths

So far we have learned two approaches to language modelling, each with its own objective:



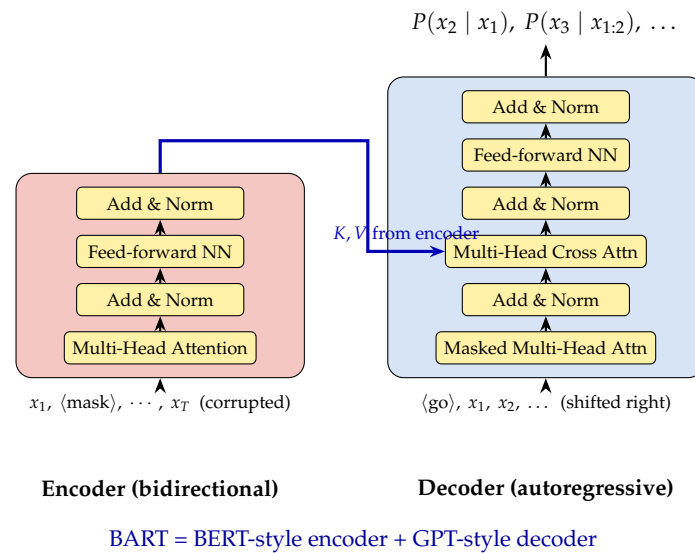
- **BERT** is good at *comprehension* tasks such as question answering because of its **bidirectional** nature: when it builds the representation of a word, it can use the words to its left and to its right.
- **GPT** is good at *text generation* because of its **unidirectional** (left-to-right) nature: training exactly mirrors how text is produced at inference, one token at a time.

Question posed in the lecture: why don't we take the best of both worlds?

1.2 BART: Bidirectional AutoRegressive Transformer

1.2.1 Architecture

BART stitches a BERT-like **encoder** (bidirectional) to a GPT-like **decoder** (autoregressive) and connects them with **cross-attention**.



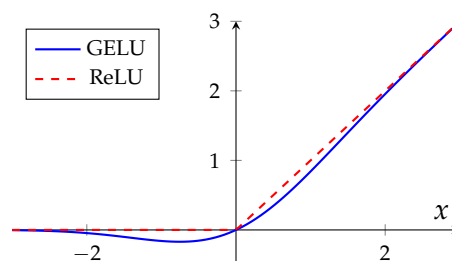
Facts from the slides:

- A **vanilla transformer** encoder-decoder architecture with the **GELU** activation function.
- The input sequence is **corrupted in multiple ways**: random masking, token deletion, document rotation, and so on.
- The **encoder** takes in the corrupted sequence.
- The **decoder** predicts the **entire original sequence** and computes the loss (as in GPT) over the entire sequence.
- One can think of this as a **denoising autoencoder**.

1.2.2 GELU activation

$$\text{GELU}(x) = x \Phi(x) \approx 0.5x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.044715x^3) \right] \right), \quad (1.1)$$

where Φ is the standard normal CDF. Unlike ReLU, it is smooth and lets small negative values through with a small weight.



Intuition : GELU

GELU weights each input by the probability that a standard Gaussian falls below it. Large positive inputs pass through ($\Phi \approx 1$), large negative inputs are suppressed ($\Phi \approx 0$), and near zero the gate is soft rather than a hard corner. The smoothness gives better-behaved gradients.

1.2.3 The noising transformations

The slide lists “random masking, token deletion, document rotation and so on”. The BART paper uses five; each is shown below on a toy document with two sentences, A B C . D E .

Original	A B C . D E .
Token masking	A [M] C . D [M] .
Token deletion	A C . D E .
Text infilling	A [M] . D [M] E .
Sentence permutation	D E . A B C .
Document rotation	C . D E . A B

- **Token masking** (as in BERT): random tokens become [MASK].
- **Token deletion**: random tokens are removed; the model must also work out *where* something is missing.
- **Text infilling**: a whole span (possibly of length 0) is replaced by a *single* [MASK]; the model must work out *how many* tokens are missing.
- **Sentence permutation**: sentences are shuffled; the model must restore the order.
- **Document rotation**: a random token is chosen and the document is rotated to start there; the model must find the true start.

1.2.4 Training objective

Let $x = (x_1, \dots, x_T)$ be the original text and $\tilde{x} = \text{noise}(x)$ the corrupted version. The encoder produces $H = \text{Enc}(\tilde{x})$ [$\tilde{T} \times d_{\text{model}}$] and the decoder models the original sequence autoregressively:

$$\mathcal{L}_{\text{BART}}(\theta) = - \sum_{i=1}^T \log P_{\theta}(x_i \mid x_{<i}, \tilde{x}) \quad (1.2)$$

Derivation : why this is the “best of both worlds”

By the chain rule of probability,

$$P(x \mid \tilde{x}) = \prod_{i=1}^T P(x_i \mid x_{<i}, \tilde{x}).$$

The factor $x_{<i}$ is handled by the causal decoder (GPT-like), and the factor $\tilde{x}$ is handled by the bidirectional encoder through cross-attention (BERT-like). Taking $-\log$ gives the loss above.

- Compare with MLM: loss only on the masked subset $\mathcal{M}$; BART has a loss on *every* position $i = 1..T$ (as in GPT).
- Compare with CLM: each prediction conditions on $x_{<i}$ only; in BART it also conditions on a (corrupted) view of the *whole* document.

Intuition : denoising autoencoder

An autoencoder learns to reconstruct its input through a bottleneck. A *denoising* autoencoder receives a damaged input and must reconstruct the clean one. To undo shuffling, deletion and masking, the model has to understand syntax, semantics and discourse, which is exactly the knowledge we want in a pre-trained model. Since the decoder generates the output, BART is directly usable for **generation** tasks (summarisation, translation), while the bidirectional encoder keeps it strong on **comprehension**.

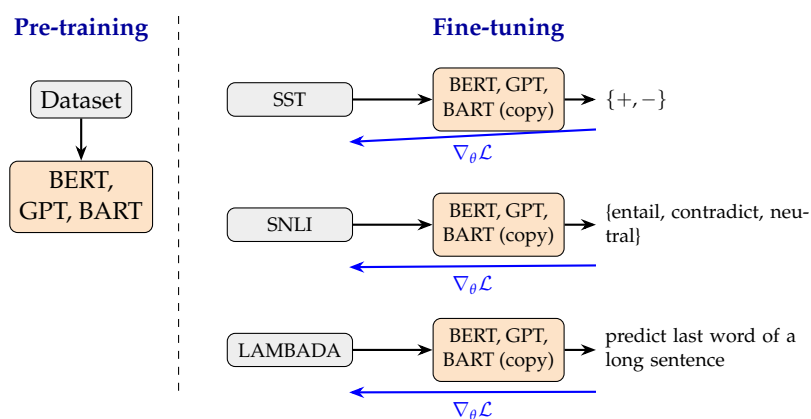
Beyond the slides : BART at a glance

BART-base: 6 encoder + 6 decoder layers; BART-large: 12 + 12 layers, $d_{\text{model}} = 1024$, about 400M parameters. The paper found text infilling (together with sentence permutation) to be the most effective noise, which foreshadows T5's *span corruption* in Module 3.

1.3 GPT-2: Prompting large language models

1.3.1 The problem with “pre-train, then fine-tune”

Using any of these pre-trained models (BERT, GPT, BART) for a downstream task requires us to **independently fine-tune** the model's parameters for each task, and the objective may differ per task. We make a *copy* of the pre-trained model for each task and fine-tune it on that task's dataset.



Problems with this recipe:

1. Fine-tuning large models is **costly**; it still needs **thousands of labelled samples** to perform well on a downstream task.
2. Some tasks may **not have enough labelled samples**.
3. It is **not how humans adapt**. Once we understand a language, we can be handed a book and *prompted* to summarise it or answer a question about it. We do not need “supervised fine-tuning” for most tasks.

Goal: a *single* model that learns to do *multiple tasks* with *zero to a few examples* instead of thousands.

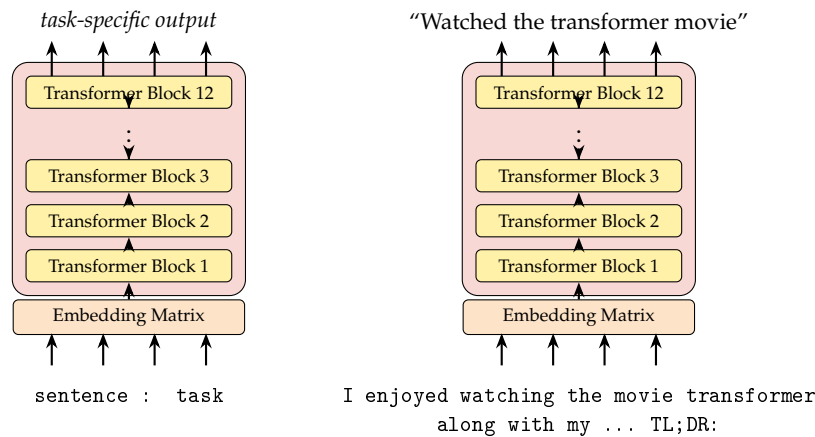
1.3.2 Zero-shot transfer by changing the input

Question: can we adapt a pre-trained LM to downstream tasks without any explicit supervision? This is called **zero-shot transfer**. **Answer:** yes, with a simple tweak to the input text.

The prompts (instructions) are themselves words, so we just change the single-task (LM) formulation

$$p(\text{output} \mid \text{input}) \longrightarrow p(\text{output} \mid \text{input}, \text{task}), \quad (1.3)$$

where the **task is a plain-language instruction** prepended (or appended) to the input sequence **at inference**. Surprisingly, this makes the model produce a task-specific response for the same input.



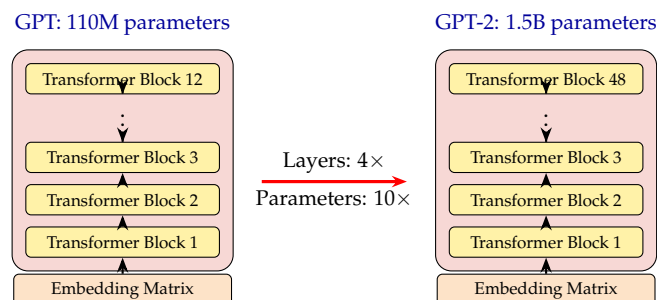
Example: Input text “*I enjoyed watching the movie transformer along with my ...*”. Task: *summarize* (or TL;DR:). Appending TL;DR: makes the most likely continuation a summary, because on the internet TL;DR: is usually followed by one.

Intuition : why prompting works at all

A language model trained on web text has seen the pattern “*long text* + TL;DR: + *short summary*” many times, the pattern “*English sentence* = *French sentence*” in bilingual blogs, and “Q: ... A: ...” in FAQs. To lower its next-token loss it has to learn these *implicit tasks*. A prompt simply puts the model into the right context so that the task’s answer becomes the most likely continuation. The task is expressed in the same space as the data (text), so no new parameters or heads are needed.

1.3.3 Scale: model and data

To get good performance we must scale both the model size and the data size.



Dataset: WebText

- A new dataset, **WebText**, created by crawling **outbound links from Reddit** with **at least 3 karma, excluding Wikipedia** links.
- Karma acts as a cheap human quality filter, which ensures **diversity and quality**. Excluding Wikipedia avoids overlap with common evaluation sets built from Wikipedia.
- **Size**: 40 GB of text, about **8 million web pages**.
- **Tokenizer**: **byte-level BPE**; **vocabulary of 50,257 tokens**.
- **What about translation?** WebText is mainly English, yet it contains *naturally occurring demonstrations* of English $\leftrightarrow$ French translation (e.g. “*I’m not the cleverest man in the world, but like they say in French: Je ne suis pas un imbécile [I’m not a fool]*”). This is where zero-shot translation ability comes from.

Beyond the slides : byte-level BPE and 50,257

Byte-level BPE starts from the 256 possible bytes, so *any* string is representable and there is never an unknown token. $50,257 = 256 \text{ base bytes} + 50,000 \text{ merges} + 1 \text{ special end-of-text token}$.

Model

GPT-2 was trained in four sizes:

Parameters	Layers	d_{model}	Setting	Value
117M	12	768	Context window	1024 tokens
345M	24	1024	Batch size	512
762M	36	1280	Objective	Causal LM (CLM)
1542M	48	1600	Vocabulary	50,257

Derivation : where does “1.5B” come from?

Per decoder block: attention $4d_{\text{model}}^2$ plus FFN $2 \cdot d_{\text{model}} \cdot 4d_{\text{model}} = 8d_{\text{model}}^2$, so $\approx 12d_{\text{model}}^2$ per layer. Adding token and position embeddings,

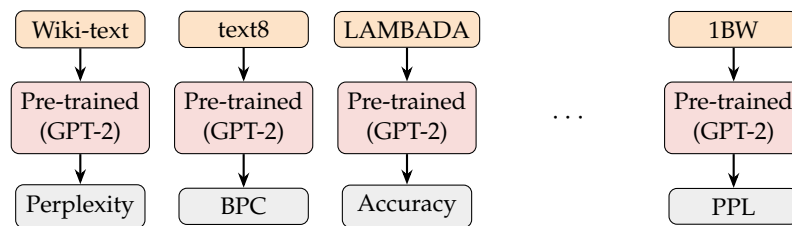
$$P \approx 12Ld_{\text{model}}^2 + Vd_{\text{model}} + Td_{\text{model}}.$$

For $L = 48$, $d_{\text{model}} = 1600$: $12 \cdot 48 \cdot 1600^2 = 1.475 \times 10^9$; $Vd_{\text{model}} = 50257 \cdot 1600 = 8.04 \times 10^7$; $Td_{\text{model}} = 1024 \cdot 1600 = 1.6 \times 10^6$. Total $\approx 1.557 \times 10^9 \approx \mathbf{1.5B}$. ✓

For $L = 12$, $d_{\text{model}} = 768$ the same formula gives $\approx 124\text{M}$. The paper reported 117M; the widely quoted corrected figure for GPT-2 small is 124M.

1.3.4 Zero-shot performance across domains

Take the pre-trained GPT-2 large (1.5B, trained on WebText) and, *without any fine-tuning*, measure it on benchmarks from different domains: LM, translation, summarisation, reading comprehension, and others.



The metrics, precisely

Perplexity (word/token-level LM, e.g. WikiText, 1BW).

$$\text{PPL} = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log P(x_i | x_{<i})\right) = \exp(\text{average cross-entropy in nats}). \quad (1.4)$$

Lower is better. A PPL of k means the model is, on average, as uncertain as a uniform choice among k tokens.

Bits per character (character-level LM, e.g. text8).

$$\text{BPC} = -\frac{1}{N_{\text{char}}} \sum_j \log_2 P(c_j | c_{<j}). \quad (1.5)$$

Lower is better. Converting: bits = nats / $\ln 2$, and $\text{PPL}_{\text{char}} = 2^{\text{BPC}}$.

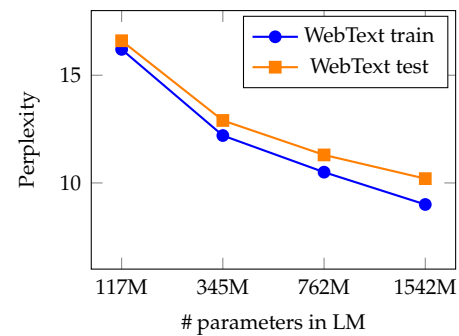
Accuracy (LAMBADA). LAMBADA is an **open-ended cloze task** with about **10,000 passages from BooksCorpus**; the model must predict the **missing final word of the last sentence** of each passage. The word can only be guessed using the *broad* context of the passage, so the benchmark tests long-range dependencies.

Results from the slides

- **WikiText (perplexity):** zero-shot GPT-2 already matches or beats earlier specialised models (WikiText-103 PPL ≈ 17.5 vs. ≈ 18.3 before), and the slide notes that **performance increases significantly upon fine-tuning** (“GPT-2 fine-tuned” is the lowest point on the Papers-with-Code chart).
- **text8 (BPC):** GPT-2 attains ≈ 0.98 BPC zero-shot, in the same range as the best character-level models (Transformer-XL, mLSTM) on the chart.
- **LAMBADA (accuracy):** GPT-2 1.5B reached $\approx 63\%$ accuracy zero-shot, above all earlier entries on the chart (e.g. the Gated-Attention Reader, just under 50%).
- **1BW (One Billion Word, PPL):** the exception. GPT-2 is *worse* than the specialised SOTA (PPL ≈ 42 vs. ≈ 22). 1BW shuffles sentences, which destroys the long-range structure GPT-2 exploits.
- **Overall:** GPT-2 **outperforms other language models trained on domain-specific data** (Wikipedia, books) **without using those domain-specific training datasets**.

1.3.5 A promising direction

- On QA, reading comprehension, summarisation and translation, zero-shot GPT-2 is **far from SOTA** models fine-tuned for those tasks.
- Even the 1.5B model **still underfits** WebText: train and test perplexities are almost equal and both keep falling with size.
- The trend suggests that scaling model and data further may make prompting much stronger (this is what GPT-3 later showed).



(trend redrawn from the slide; values approximate)

Key takeaway

This work established that “**Large Language Models are Unsupervised Multitask Learners**”. A plain CLM trained on a large, diverse corpus picks up many tasks implicitly, and they can be invoked with a natural-language prompt, with *no* gradient updates.

Intuition : underfitting is good news here

Overfitting means the model has already squeezed everything it can from the data; underfitting at 1.5B parameters means *more capacity would still help*. That is the empirical seed of the scaling-law era (GPT-3 at 175B).

Module 2

Choices that affect performance

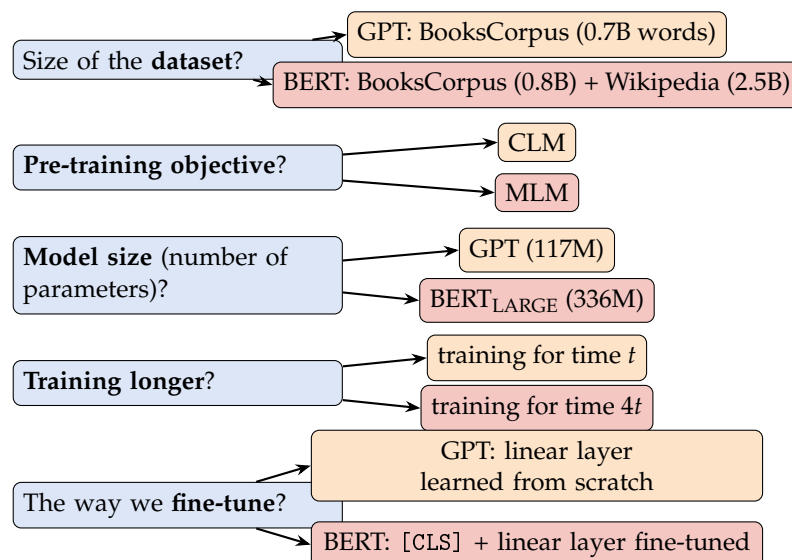
2.1 Why does one model beat another?

Compare GPT (about 110M parameters) and BERT on the GLUE benchmark:

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average –
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

(The second header row gives the number of training examples per task.)

BERT_{BASE} beats GPT on every task, even though the two have almost the same size. **Why?** Many things differ between the two pipelines, and each is a candidate explanation:



Note on the slides

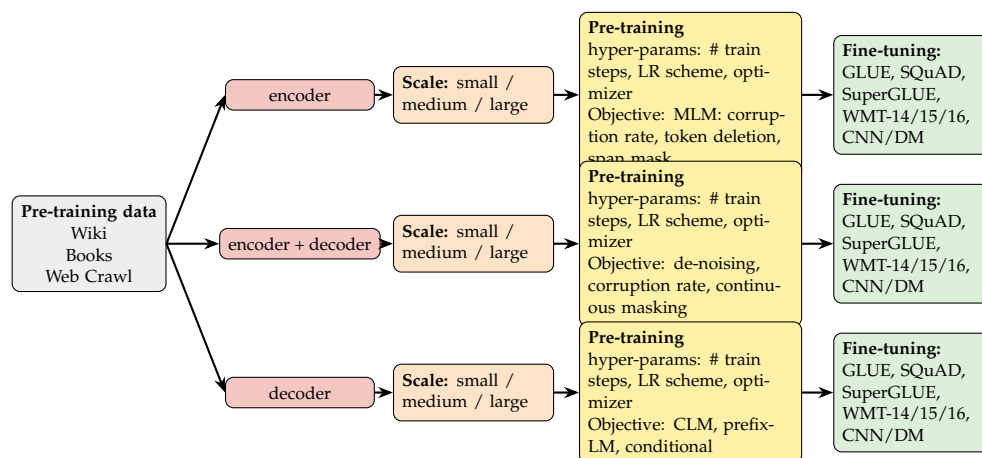
The slide says “BookCorpus (1B)” for BERT and “Books (0.7B)” for GPT. The original papers report about 800M words for BooksCorpus and 2.5B for English Wikipedia. The point is the *ratio*: BERT saw roughly $4.5\times$ more text.

Intuition : confounding variables

This is a classic confounded experiment: BERT differs from GPT in data, objective, size, training time and fine-tuning recipe *all at once*. When two things differ in five ways, a single score cannot tell you which difference caused the gap. That is why the rest of the week is a controlled study.

2.2 The design space

To answer these questions we need extensive experiments that **keep one aspect of the pipeline constant and vary the other**. The design space is large:



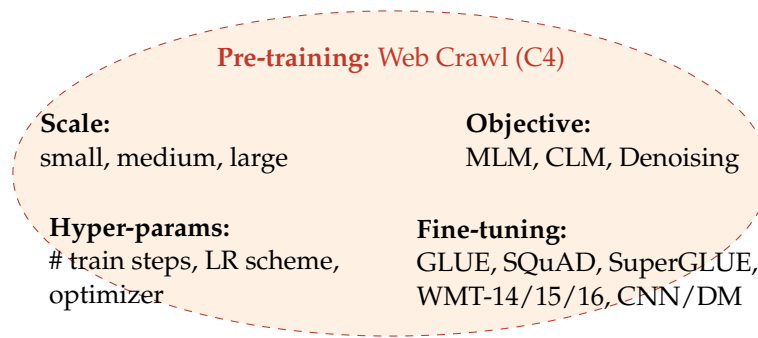
A full grid over these knobs is combinatorially huge. Every knob interacts with the others, and every experiment costs GPU-weeks.

2.3 Wishlist

1. Can we have the **same objective for both pre-training and fine-tuning**? (BERT pre-trains with MLM but fine-tunes with a classification head, and the two do not match.)
2. Can we have an **architecture that uses both CLM- and MLM-like objectives**?
3. Can we use a pre-training dataset that is **as large as possible**, to understand the effect of the size of unlabelled data?

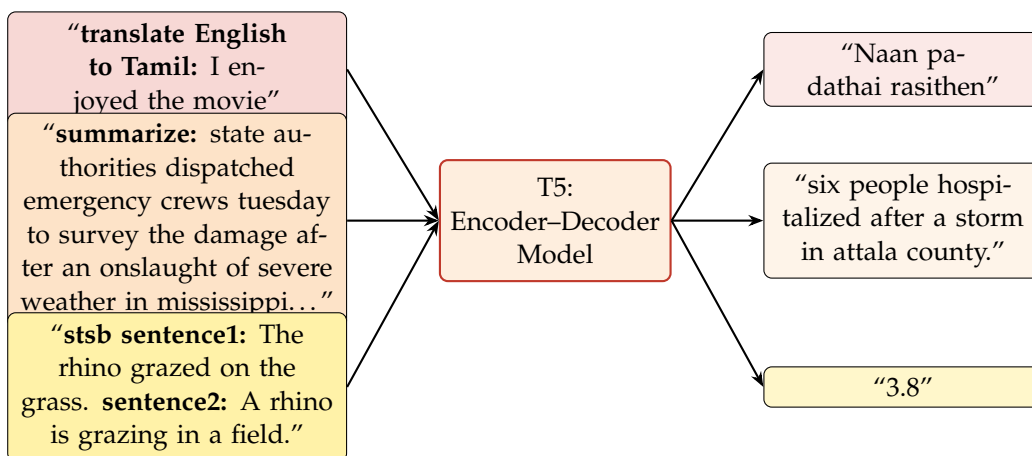
2.4 A systematic study: T5

The T5 paper (*Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*, Raffel et al. 2019) does exactly this: compare many approaches on a **diverse set of tasks** while keeping as **many factors fixed as possible**.



2.4.1 The key idea: everything is text-to-text

Formulate every NLP problem as “text in, text out”. This is the **Text-To-Text Transfer Transformer (T5)**: an encoder-decoder model whose input carries a **task-specific prefix**.



Intuition : why text-to-text is so powerful

- **One model, one loss, one decoding procedure** for every task: maximum likelihood on the target text with teacher forcing. No task-specific heads.
- Even **regression** becomes text: STS-B similarity scores (0–5) are rounded to the nearest 0.2 and emitted as strings such as “3.8”.
- **Classification** becomes emitting the label word (“entailment”, “positive”).
- The **same objective** is used for pre-training and fine-tuning (maximum likelihood of target text), which ticks wishlist item 1. The encoder-decoder can emulate CLM, MLM or prefix-LM just by changing masks, which ticks item 2.
- Since every task shares one format, we can compare architectures, objectives and datasets *fairly*, which is the whole point of the study.

Derivation : the text-to-text training loss

For an (input, target) pair (x, y) with $y = (y_1, \dots, y_m)$,

$$\mathcal{L}(\theta) = - \sum_{j=1}^m \log P_{\theta}(y_j | y_{<j}, x), \quad P_{\theta}(\cdot | \cdot) = \text{softmax}(h_j^{\text{dec}} W_{\text{emb}}^{\top}) \quad [V]$$

where h_j^{dec} $[d_{\text{model}}]$ is the final decoder state and W_{emb} $[V \times d_{\text{model}}]$ is the (shared) embedding matrix. The loss is identical whether x is a corrupted C4 passage (pre-training) or “summarize: ...” (fine-tuning).

2.4.2 Dataset: the Colossal Clean Crawled Corpus (C4)

Property	Value
Source	Common Crawl web scrape, heavily filtered
Language	Mainly English
Tokenizer	SentencePiece
Vocabulary size	32,000
Size	750 GB, about 300M documents
Number of tokens	156 billion (about $52\times$ the data used for pre-training BERT)

Beyond the slides : what “clean” means in C4

Filters applied to Common Crawl include: keep only lines ending in terminal punctuation; drop pages with fewer than 3 sentences and lines with fewer than 5 words; remove pages containing words from a “bad words” list; remove lines with “Javascript” and pages with “lorem ipsum” or code braces `{}`; deduplicate any three-sentence span occurring more than once; keep pages that `langdetect` classifies as English with probability ≥ 0.99 . The lecture says the details of building such a dataset come in later lectures.

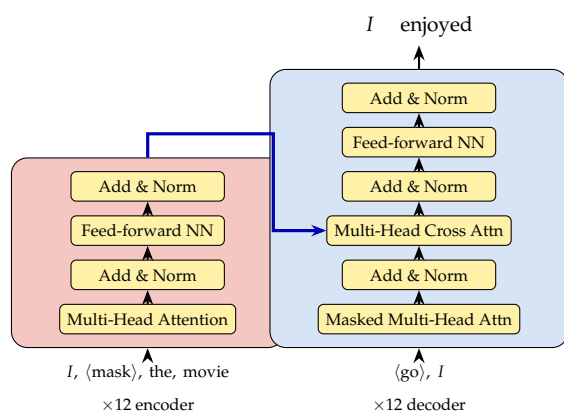
Derivation : quick sanity arithmetic on C4

- Bytes per token: $750 \times 10^9 / 156 \times 10^9 \approx 4.8$ bytes/token, about one sub-word unit of English.
- Tokens per document: $156 \times 10^9 / 300 \times 10^6 \approx 520$, about one 512-token context.
- “ $52\times$ BERT”: BERT’s corpus was $\approx 3.3\text{B}$ words; $156/3.3 \approx 47$, i.e. roughly $50\times$.

Module 3

Pre-training and fine-tuning a baseline model

3.1 The baseline architecture



- An encoder–decoder model like BART.
- An encoder–decoder lets us **experiment with all possible architectures and objectives**: switching masks turns it into an encoder-only, decoder-only or prefix-LM model.
- **Baseline**: 12 encoder and 12 decoder blocks, each stack equivalent in spirit to BERT_{base} (12 layers, $d_{\text{model}} = 768$, 12 heads, $d_{\text{ff}} = 3072$).
- About **220M parameters**, twice BERT_{base} (110M).

Derivation : why $\approx 220\text{M}$

With $d_{\text{model}} = 768$, $d_{\text{ff}} = 3072$, $V = 32,000$:

$$\text{encoder layer} = \underbrace{4d_{\text{model}}^2}_{\text{self-attn}} + \underbrace{2d_{\text{model}}d_{\text{ff}}}_{\text{FFN}} = 2.36\text{M} + 4.72\text{M} = 7.08\text{M}$$

$$\text{decoder layer} = \underbrace{4d_{\text{model}}^2}_{\text{self}} + \underbrace{4d_{\text{model}}^2}_{\text{cross}} + 2d_{\text{model}}d_{\text{ff}} = 9.44\text{M}$$

$$12 \text{ enc} + 12 \text{ dec} = 12(7.08 + 9.44)\text{M} \approx 198\text{M}$$

$$\text{embeddings} = Vd_{\text{model}} = 32000 \times 768 \approx 24.6\text{M} \quad (\text{shared: enc, dec, output})$$

$$\text{total} \approx 223\text{M} \approx \mathbf{220\text{M}}.$$

3.2 The denoising objective: span corruption

Analogous to MLM, but we **replace a random span of tokens by a single sentinel token**. Each span gets its own unique sentinel (<w>, <x>, <y>, ...). The target lists each sentinel followed by the tokens it hid, and ends with a final sentinel.

Original: What sets this ~~mission apart~~ is the pivotal role of artificial ~~intelligence~~ (AI) in guiding the spacecraft during its ~~critical descent~~ to the moon's surface.

Input: What sets this <w> is the pivotal role of artificial <x> (AI) in guiding the spacecraft during its <y> to the moon's surface.

Target: <w> mission apart <x> intelligence <y> critical descent <z>

Key takeaway

One advantage of this objective is that it **reduces the computational cost** and therefore the **training time**: the target is short (only the dropped spans plus sentinels) instead of the whole sequence, and the input also shrinks because each span collapses into one token.

Derivation : how much shorter are the sequences?

Let the input have n tokens, corruption rate r and mean span length ℓ . Then

$$\text{\#corrupted tokens} = rn, \quad \text{\#spans } s = \frac{rn}{\ell}.$$

Encoder input length = $n - rn + s$ (drop the corrupted tokens, add one sentinel per span).

Decoder target length = $rn + s + 1$ (hidden tokens, one sentinel per span, one final sentinel).

With $n = 512, r = 0.15, \ell = 3$: $rn = 76.8 \approx 77, s \approx 26$.

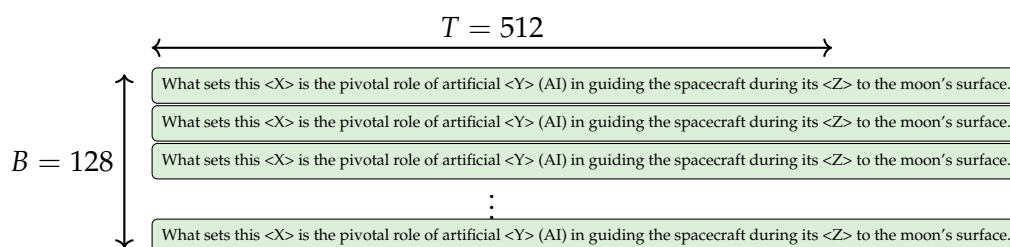
$$\text{input} \approx 512 - 77 + 26 = 461, \quad \text{target} \approx 77 + 26 + 1 = 104.$$

A BART-style objective that reconstructs the whole sequence would need a 512-token target. Decoder cost scales with target length, so the decoder does about $104/512 \approx 20\%$ of the work.

Intuition : span corruption vs. MLM vs. BART

- **MLM (BERT):** one [MASK] per token; predict each masked token independently. The spans are not “real” units.
- **BART-style:** reconstruct the whole text. Expensive, since most target tokens are copies.
- **Span corruption (T5):** hide contiguous spans and predict only the hidden parts, in order. The target is short and the model must generate multi-token spans, which is closer to real generation.

3.3 Pre-training setup



- Context length $T = 512$; batch size $B = 128$ sequences.

- Tokens per batch $\approx 512 \times 128 = 65,536 = 2^{16}$.
- Number of training steps $= 2^{19} = 524,288$.
- Total pre-training tokens $= 2^{19} \times 2^{16} = 2^{35} \approx \mathbf{34B}$.

Derivation : the powers of two

$512 = 2^9$, $128 = 2^7$, so tokens per batch $= 2^{9+7} = 2^{16}$. Then total tokens $= 2^{19} \cdot 2^{16} = 2^{35}$. Since $2^{10} \approx 10^3$, $2^{35} = 2^5 \cdot 2^{30} \approx 32 \times 1.07 \times 10^9 \approx 3.4 \times 10^{10} = 34B$.

This is only a **fraction of C4**: $34/156 \approx 22\%$. We have not done even one epoch, so **no example is ever repeated**. For comparison, BERT was trained on 137B tokens and RoBERTa on 2.2T. For the given compute budget, and to be able to try many hyperparameters, 34B tokens is a reasonable size for a baseline.

3.3.1 Optimizer: AdaFactor

AdaFactor needs **less memory than Adam**.

Derivation : why AdaFactor saves memory

Adam keeps two extra numbers per parameter: the first moment m and the second moment v . For a weight matrix W $[n \times m]$ that is $2nm$ extra floats.

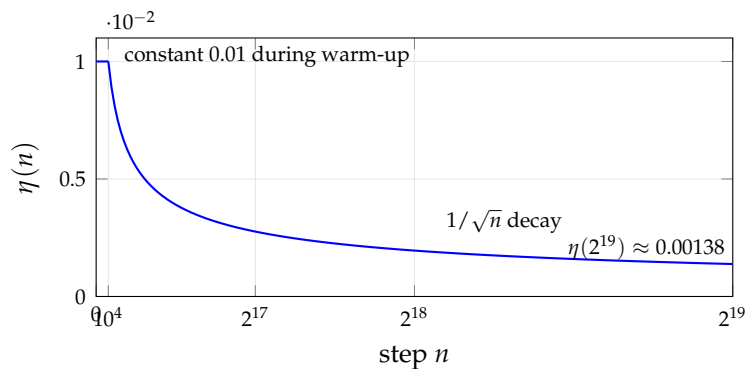
AdaFactor (i) drops the first moment (or makes it optional) and (ii) *factorises* the second moment: it stores only row sums R $[n]$ and column sums C $[m]$ of the squared gradients and reconstructs

$$\hat{V}_{ij} \approx \frac{R_i C_j}{\sum_k R_k} \quad \Rightarrow \quad n + m \text{ floats instead of } nm.$$

For an FFN matrix 768×3072 : Adam needs $2 \times 2,359,296 = 4,718,592$ extra floats; AdaFactor needs $768 + 3072 = 3,840$.

3.3.2 Learning-rate schedule: inverse square root

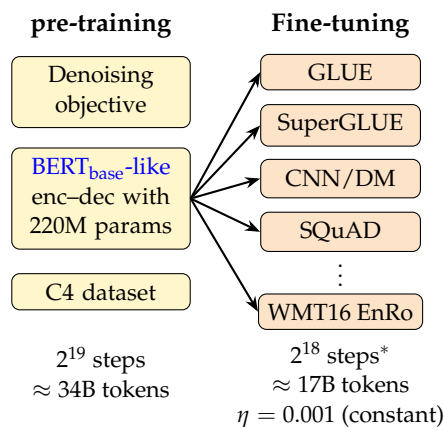
$$\eta(n) = \frac{1}{\sqrt{\max(n, k)}}, \quad n = \text{training step}, k = 10^4 \text{ warm-up steps.} \quad (3.1)$$



Intuition : reading the schedule

For $n < k$, $\max(n, k) = k$, so $\eta = 1/\sqrt{10^4} = 0.01$ is constant (T5 holds a constant rate here rather than ramping up). After that η decays as $1/\sqrt{n}$: large early updates explore; later updates shrink so the model settles. Doubling the step count divides the rate by $\sqrt{2}$.

3.4 Fine-tuning setup



GLUE / SuperGLUE: collections of tasks (CoLA, SST, STSB, MNLI, ..., RTE) for natural language understanding; report the average score defined by their authors.

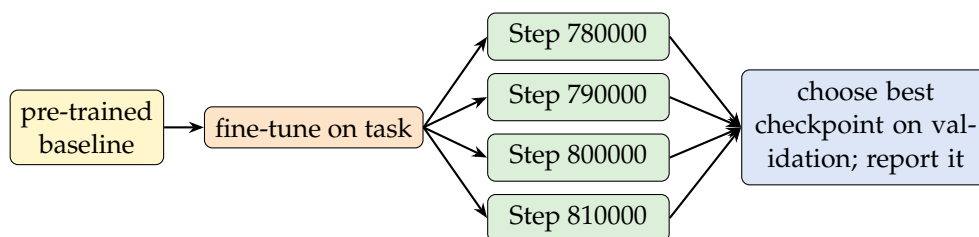
CNN/DM: text summarisation.

SQuAD: question answering.

WMT16 EnRo (and WMT14 EnDe, EnFr): translation.

Metric depends on the task: **BLEU** for translation, **ROUGE** for summarisation, **Exact-Match / F1** for QA.

- Fine-tuning uses 2¹⁸ steps with the **same number of tokens per batch** (2¹⁶), but with $B = 512$ and $T = 128$ (task examples are short). Total $2^{18} \times 2^{16} = 2^{34} \approx 17\text{B}$ tokens.
- *The step count is a **trade-off** between high-resource tasks (which benefit from longer fine-tuning) and low-resource tasks (which overfit).
- Constant learning rate $\eta = 0.001$.
- Checkpoint selection:** evaluate on the validation set every 5,000 steps (... , 780000, 790000, 800000, 810000, ...) and **report the best checkpoint** for each task.

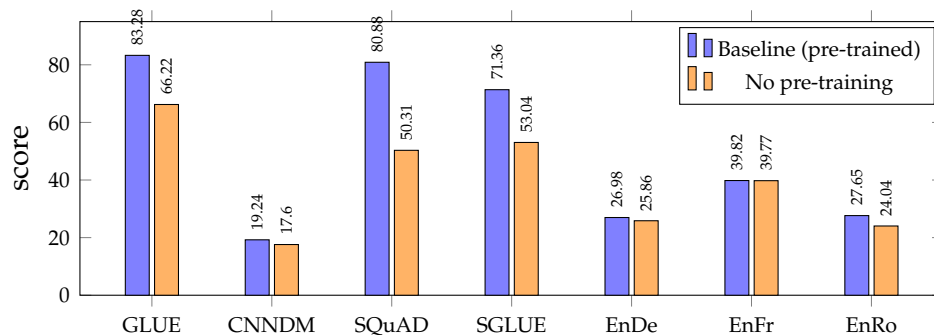


In what follows, ★ marks the baseline model in every table.

3.5 Baseline performance

The baseline is pre-trained and fine-tuned **10 times from scratch** with different initialisations and data shuffling, to get a mean and a standard deviation. The model is also **trained directly on the benchmarks** (no pre-training) for 2¹⁸ steps, the same number as fine-tuning.

	GLUE	CNN3M	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Baseline average	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Baseline standard deviation	0.235	0.065	0.343	0.416	0.112	0.090	0.108
No pre-training	66.22	17.60	50.31	53.04	25.86	39.77	24.04



Key takeaway

- Unsurprisingly, **pre-training helps** across a variety of tasks, most of all on **small datasets** (SQuAD +30.6, GLUE +17, SuperGLUE +18).
- With a **big supervised dataset** (WMT English→French, about 40M sentence pairs) supervised training alone nearly matches pre-training (39.77 vs. 39.82, within one standard deviation).
- The standard deviation row tells us how big a difference must be before we trust it. A gap smaller than about 2σ (e.g. 0.5 GLUE points) is noise.

3.5.1 Is this comparable to BERT?

BERT_{base} achieved **80.8** on SQuAD and **84.4** on MNLI; the baseline achieved **80.88** on SQuAD and **84.24** on MNLI. However, **we cannot directly compare** the two: the architectures and parameter counts differ, and BERT_{base} was trained on 137B tokens while the baseline saw about 34B. **Therefore, for a given task, we need a principled way to compare different architectures**, which is the subject of Module 4.

Note on the slides

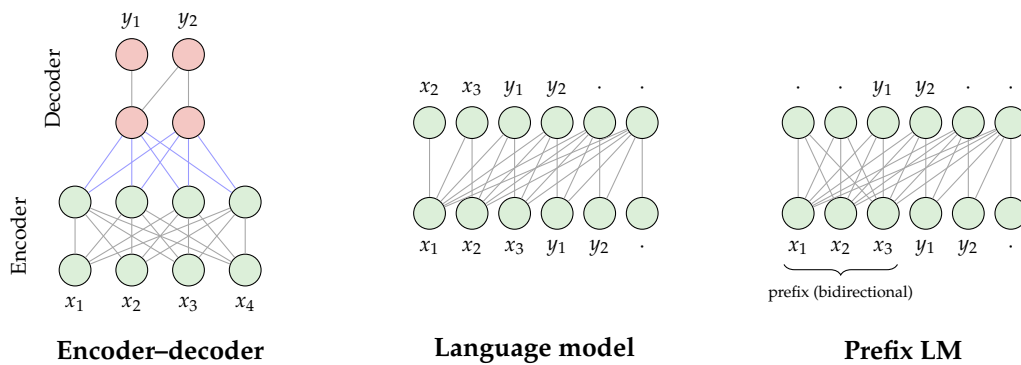
One slide says the baseline was trained on “37B tokens”; everywhere else it is $2^{35} \approx 34\text{B}$. Use 34B.

Module 4

Experimenting with different architectures

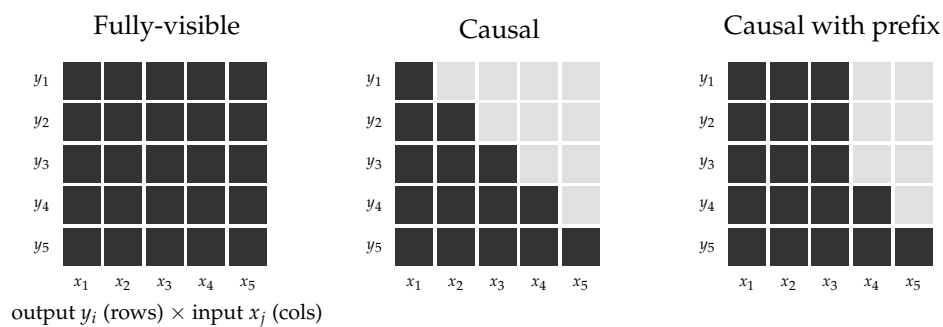
The three architectural variants

The study compares three architectures, each of which can be trained with a denoising (MLM-like) or LM objective:



Lines show which lower-layer positions each upper-layer position may attend to (blue lines = cross-attention).

Mask patterns



Dark cells: self-attention allowed. **Light cells:** self-attention not allowed.

- **Fully-visible:** every output position sees every input position. Used in the encoder.
- **Causal:** output i sees inputs $1..i$ (lower triangle). Used in a decoder / LM.

- **Causal with prefix:** positions inside the prefix (here x_1, x_2, x_3) are visible to *everyone*; beyond the prefix the mask is causal. This lets one decoder-only stack behave like an encoder on the prefix and a decoder on the rest.

Derivation : writing the masks as matrices

With query index i , key index j (1-based) and prefix length p :

$$\mathcal{M}_{ij}^{\text{full}} = 0, \quad \mathcal{M}_{ij}^{\text{causal}} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}, \quad \mathcal{M}_{ij}^{\text{prefix}} = \begin{cases} 0 & j \leq \max(i, p) \\ -\infty & \text{otherwise} \end{cases}$$

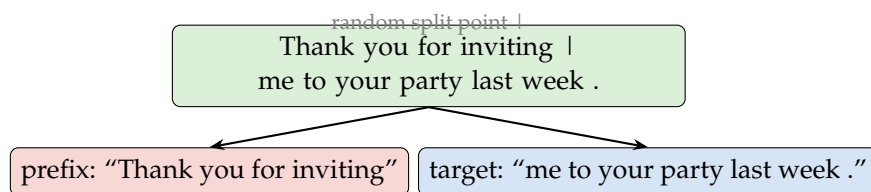
Number of allowed cells for sequence length n : full = n^2 ; causal = $n(n+1)/2$; prefix = $p^2 + \sum_{i=p+1}^n i = p^2 + \frac{n(n+1)}{2} - \frac{p(p+1)}{2}$. For $n = 5, p = 3$: $9 + 15 - 6 = 18$ (count the dark cells above). Note $p = 0$ gives causal and $p = n$ gives full: **prefix LM interpolates between the two.**

Prefix language modelling

Consider the task-specific input “*summarize the following: I enjoyed watching the transformer movie along with my ...*”.

- In prefix-LM we want the **input sequence to be attended bidirectionally** and the **task output sequence autoregressively**.
- So we can use the **encoder for the input** and the **decoder for the output** (the encoder-decoder), **or**
- use a **single decoder with the prefix mask** instead of the causal mask, prepending the task to the input sequence.
- **Pre-training on C4 with prefix-LM:** take a random span of text and split it at a random point into a **prefix** (input) and a **target**.

The prefix-LM objective was also used to pre-train a model in a unified text-to-text framework (UniLM).



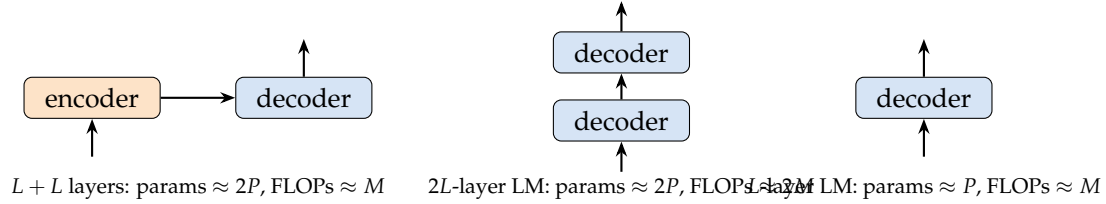
Comparing architectures fairly: parameters vs. FLOPs

The baseline is an encoder-decoder. How do we compare it with encoder-only or decoder-only models? We must show that the models are *equivalent in some meaningful way*. Two natural criteria:

1. the **number of parameters** P ;
2. the **amount of compute** M (FLOPs) needed to process an input and target sequence.

But we cannot satisfy both at the same time.

- An encoder-decoder with L encoder layers and L decoder layers has **approximately the same number of parameters** as a language model with $2L$ layers.
- But its **compute cost** equals that of an L -layer language model for the same input sequence and task.



Derivation : the exercise from the slide, “How? take it as an exercise problem”

Let one stack of L layers have P parameters. A forward pass costs about 2 FLOPs per parameter per token (one multiply and one add), so

$$\text{FLOPs} \approx 2 \times (\text{parameters that touch the token}) \times (\text{number of tokens}).$$

Let the input have n_x tokens and the target n_y tokens.

Parameters. Encoder (L layers) $\approx P$, decoder (L layers) $\approx P \Rightarrow \text{enc-dec} \approx 2P$. A $2L$ -layer LM also has $\approx 2P$. ✓ Same parameters.

FLOPs of enc-dec. Encoder parameters touch only the n_x input tokens; decoder parameters touch only the n_y target tokens:

$$M_{\text{enc-dec}} \approx 2P n_x + 2P n_y = 2P(n_x + n_y).$$

FLOPs of an L -layer LM. A decoder-only LM must process the concatenation $[x; y]$ of length $n_x + n_y$ through all its P parameters:

$$M_{L\text{-LM}} \approx 2P(n_x + n_y) = M_{\text{enc-dec}}.$$

FLOPs of a $2L$ -layer LM. Now $2P$ parameters touch all $n_x + n_y$ tokens: $M_{2L\text{-LM}} \approx 2 \cdot 2P(n_x + n_y) = 2M$.

Conclusion. Matching parameters (enc-dec vs. $2L$ -LM) doubles the LM’s compute, and matching compute (enc-dec vs. L -LM) halves the LM’s parameters. You cannot match both, so the table reports both P and M for every row.

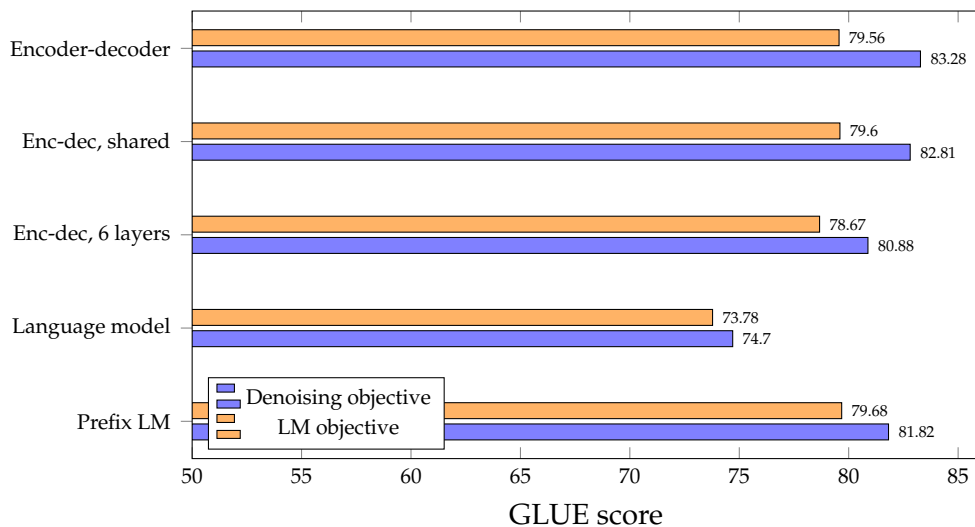
Watch out : “approximately” the same parameters

A decoder layer has an *extra* cross-attention block ($4d_{\text{model}}^2$). With $d_{\text{ff}} = 4d_{\text{model}}$: encoder layer = $4d_{\text{model}}^2 + 8d_{\text{model}}^2 = 12d_{\text{model}}^2$, decoder layer = $16d_{\text{model}}^2$. So $L + L$ enc-dec = $28Ld_{\text{model}}^2$ while a $2L$ -layer LM = $24Ld_{\text{model}}^2$, about 17% fewer. Hence “approximately”.

Results: architecture $\times$ objective

P : number of parameters in the BERT_{base}-sized model (**not the baseline**). M : FLOPs for an $L + L$ layer enc-dec model or an L -layer decoder-only model. The first row is the baseline, which has **twice** the parameters of BERT_{base}.

Architecture	Objective	Params	Cost	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Encoder-decoder	Denoising	$2P$	M	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Enc-dec, shared	Denoising	P	M	82.81	18.78	80.63	70.73	26.72	39.03	27.46
Enc-dec, 6 layers	Denoising	P	$M/2$	80.88	18.97	77.59	68.42	26.38	38.40	26.95
Language model	Denoising	P	M	74.70	17.93	61.14	55.02	25.09	35.28	25.86
Prefix LM	Denoising	P	M	81.82	18.61	78.94	68.11	26.43	37.98	27.39
Encoder-decoder	LM	$2P$	M	79.56	18.59	76.02	64.29	26.27	39.17	26.86
Enc-dec, shared	LM	P	M	79.60	18.13	76.35	63.50	26.62	39.17	27.05
Enc-dec, 6 layers	LM	P	$M/2$	78.67	18.26	75.32	64.06	26.13	38.42	26.89
Language model	LM	P	M	73.78	17.54	53.81	56.51	25.23	34.31	25.38
Prefix LM	LM	P	M	79.68	17.84	76.87	64.86	26.28	37.51	26.76



The lecture walks through the rows one at a time:

1. **Enc-dec, shared** ($\theta_{\text{Enc}} = \theta_{\text{Dec}}$, 12+12 layers, P params, M cost): only a **slight decrease** in performance from sharing parameters, despite halving the parameter count (GLUE 83.28 $\rightarrow$ 82.81).
2. **Enc-dec, 6 layers** (6+6, P params, $M/2$ cost): **decreasing the number of layers degrades** downstream performance noticeably (GLUE 80.88, SQuAD 77.59).
3. **Language model** (12-layer decoder only, P , M): for fixed P and M the **enc-dec does better than the LM**. The causal LM is the worst row (SQuAD 61.14).
4. **Prefix LM** (12-layer decoder with prefix mask, P , M): for fixed P and M , the decoder-only prefix LM is **similar to the enc-dec** (GLUE 81.82 vs. shared 82.81). The bidirectional prefix is what matters.
5. **Changing the objective** (bottom block): the **denoising objective always gives better downstream performance than the LM objective**, for every architecture.

Intuition : why sharing barely hurts but shrinking does

Sharing weights halves the parameters *but keeps the computation*: each token still passes through 12 layers of transformations (the same weights are reused as encoder and decoder). Halving the layers halves both parameters *and* depth; depth, i.e. the number of sequential transformations, is what builds rich representations. So **compute/depth matters more than raw parameter count** at this scale.

Why is the plain LM so bad on SQuAD? A causal model cannot let the question tokens attend to the passage tokens that come after them, so it misses bidirectional context exactly where comprehension needs it. The prefix LM fixes this by making the input fully visible.

Key takeaway

Best architecture: **encoder-decoder** (a prefix-LM decoder is a close second). Best objective: **denoising**. Parameter sharing between encoder and decoder is nearly free.

4.1 Module 4.1: Effect of unsupervised objectives

4.1.1 Example (input, target) pairs for each objective

Input text: "Thank you for inviting me to your party last week ."

Objective	Inputs	Targets
Prefix language modelling	Thank you for inviting	me to your party last week .
Deshuffling	party me for your to . last fun you inviting week Thank	(original text)
BERT-style*	Thank you <M> <M> me to your party apple week .	(original text)
MASS-style**	Thank you <M> <M> me to your party <M> week .	(original text)
⋮		
★ Random spans	Thank you <X> to <Y> week .	<X> for inviting me <Y> your party last <Z>

*Slightly different from the original BERT objective, where only the masked tokens are predicted; here the decoder reconstructs the whole original text. **MASS-style is BERT-style *without* random replacement of tokens (only <M>s, no "apple"-type swaps).

Intuition : the three families

- **Language modelling (prefix-LM):** predict the continuation from a prefix.
- **Deshuffling:** the input is a random permutation of the tokens; reconstruct the original order.
- **BERT-style (denoising):** corrupt some tokens (mask/replace); reconstruct. Variants differ in *how* to corrupt and *what* the target is.

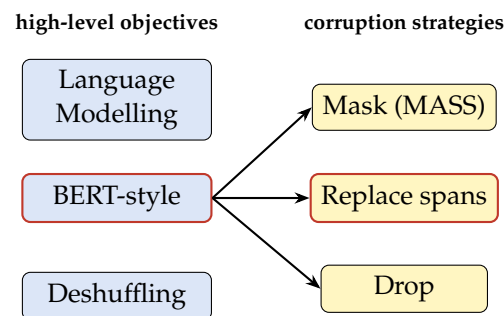
4.1.2 High-level objectives

Objective	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
Prefix language modelling	80.69	18.94	77.99	65.27	26.86	39.73	27.49
BERT-style (Devlin et al., 2018)	82.96	19.17	80.65	69.85	26.78	40.03	27.41
Deshuffling	73.17	18.59	67.61	58.47	26.11	39.30	25.62

- Both prefix-LM and BERT-style are **better on translation**.
- The **deshuffling objective is worse**.
- **Overall, a BERT-style objective is better**.

Other ways to corrupt the input exist: continuous masking (MASS), replacing random spans of text, dropping tokens. Which is best?

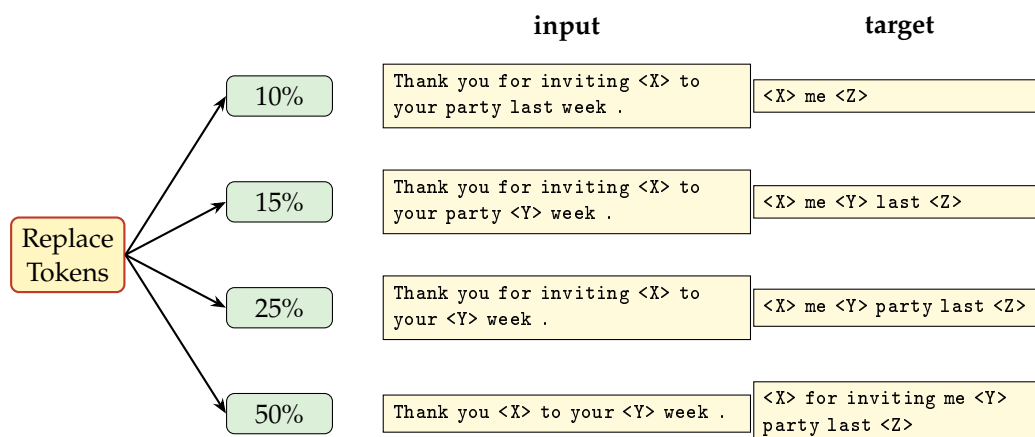
4.1.3 Corruption strategies



Objective	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
BERT-style (Devlin et al., 2018)	82.96	19.17	80.65	69.85	26.78	40.03	27.41
MASS-style	82.32	19.16	80.10	69.28	26.79	39.89	27.55
★ Replace corrupted spans	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Drop corrupted tokens	84.44	19.31	80.52	68.67	27.07	39.76	27.82

- **All these variants perform similarly.**
- “Dropping corrupted tokens” is slightly better on GLUE, because it did very well on one sub-task (CoLA), but it is **worse on SuperGLUE**.
- **Overall, replacing corrupted spans with a sentinel token performs well** and also gives short targets, so it is chosen: Thank you <X> to <Y> week . → <X> for inviting me <Y> your party last <Z>

4.1.4 Corruption rate

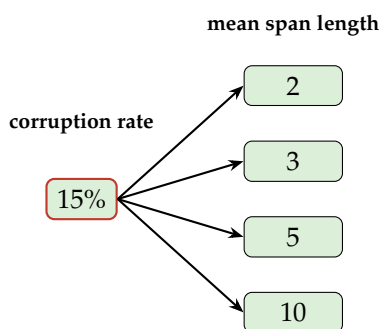


Corruption rate	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
10%	82.82	19.00	80.38	69.55	26.87	39.28	27.44
★ 15%	83.28	19.24	80.88	71.36	26.98	39.82	27.65
25%	83.00	19.54	80.96	70.48	27.04	39.83	27.47
50%	81.27	19.32	79.80	70.33	27.01	39.90	27.49

Key takeaway

Corruption rate has minimal effect on performance (10–25% are all similar; 50% hurts GLUE and SQuAD slightly). 15% is kept. A higher rate also gives *longer targets*, which slows training.

4.1.5 Corrupted span length



Baseline (i.i.d.): each token is corrupted independently; consecutive corrupted tokens are merged into one span, which gives an average span length of about 1.2 at 15%. The other rows force a *specific average* span length.

Span length	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Baseline (i.i.d.)	83.28	19.24	80.88	71.36	26.98	39.82	27.65
2	83.54	19.39	82.09	72.20	26.76	39.99	27.63
3	83.49	19.62	81.84	72.53	26.86	39.65	27.62
5	83.40	19.24	82.05	72.23	26.88	39.40	27.53
10	82.85	19.33	81.84	70.44	26.79	39.49	27.69

Key takeaway

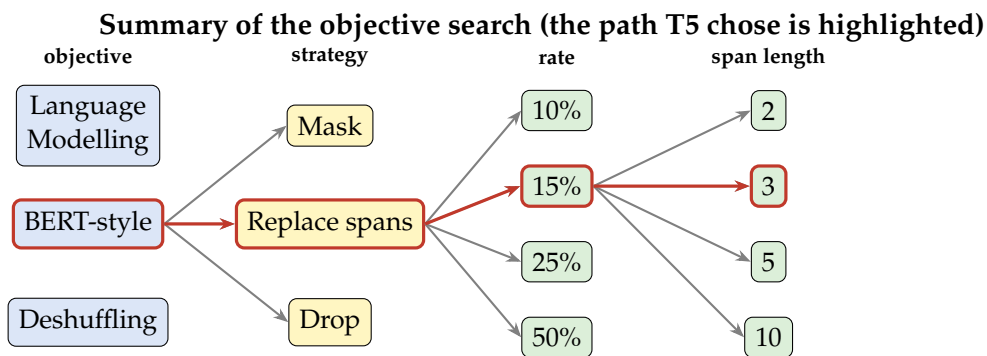
There is little difference in performance, except that span length 10 underperforms on two tasks. **A span length of 3 outperforms the others on non-translation tasks** (best CNNDM and SGLUE). The final T5 uses 15% corruption with mean span length 3.

Derivation : why longer spans also speed things up

From the formula in Module 3, the number of spans is $s = rn/\ell$. For $n = 512$, $r = 0.15$:

$$\ell \approx 1.2 \text{ (i.i.d.)} : s \approx 65, \quad \ell = 3 : s \approx 26, \quad \ell = 10 : s \approx 8.$$

Fewer spans means fewer sentinels, so both input and target get shorter. The target length $rn + s + 1$ drops from ≈ 143 to ≈ 104 (going from $\ell \approx 1.2$ to $\ell = 3$).



4.2 Module 4.2: Effect of the size of the pre-training dataset

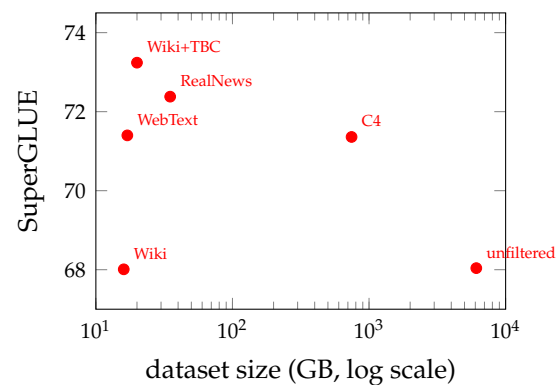
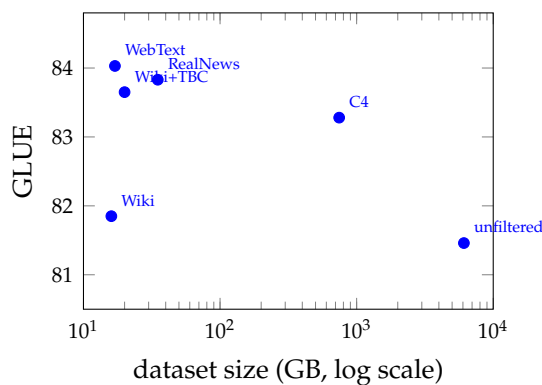
4.2.1 Quality vs. quantity

Do the **scale** and **quality** of the pre-training dataset determine downstream performance? Start from C4 (clean/filtered) and compare:

- **C4, unfiltered:** only language-detection filtering; no deduplication, bad-word removal, etc. Almost **8× bigger** than C4 (6.1TB).
- **RealNews-like:** C4 filtered to news websites, as in the RealNews dataset. About **21× smaller** than C4 (35GB).
- **WebText-like:** built like GPT-2’s WebText (C4 pages whose URL appeared on Reddit with score ≥ 3).
- **Wikipedia:** English Wikipedia only (the slide calls it “a pre-training dataset used in GPT”; the original GPT actually used BooksCorpus).
- **Wikipedia + BookCorpus (TBC):** the pre-training data used in BERT.

The last three are about **42× smaller** than C4.

Dataset	Size	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ C4	745GB	83.28	19.24	80.88	71.36	26.98	39.82	27.65
C4, unfiltered	6.1TB	81.46	19.14	78.78	68.04	26.55	39.34	27.21
RealNews-like	35GB	83.83	19.23	80.39	72.38	26.75	39.90	27.48
WebText-like	17GB	84.03	19.31	81.42	71.40	26.80	39.74	27.59
Wikipedia	16GB	81.85	19.31	81.29	68.01	26.94	39.69	27.67
Wikipedia + TBC	20GB	83.65	19.28	82.08	73.24	26.77	39.63	27.57



Size alone does not predict performance: the biggest dataset (unfiltered, 6.1TB) is among the worst.

Key takeaway

- **Unfiltered C4 degraded performance across all tasks despite being $8\times$ larger. Quality matters.**
- **RealNews-like**, despite being $21\times$ smaller, performs well across many tasks.
- **Pre-training on in-domain unlabelled data helps:** Wikipedia + TBC gives the best SuperGLUE (73.24), largely because SuperGLUE's MultiRC is built from fiction and encyclopedic text, and SQuAD is built from Wikipedia (82.08).

Watch out : the hidden cost of small in-domain datasets

Wikipedia + TBC is only 20GB. If you train on 34B tokens with it you must *repeat* the data many times, which is the subject of the next experiment.

4.2.2 Does repeating data hurt?

Questions to ponder:

- Do we need all 156B tokens for pre-training?
- The baseline saw only a fraction (34B) of C4, i.e. each example exactly once. Would **repeating examples** help or hurt downstream performance?

Experiment: keep the total at 2^{35} tokens but vary the size of the unique dataset from 2^{23} to 2^{29} tokens, so it is repeated $2^{35}/2^k$ times.

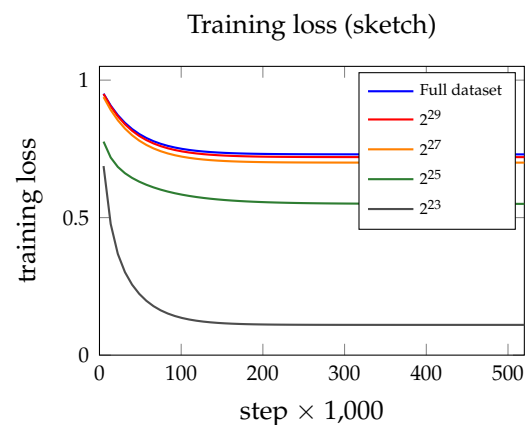
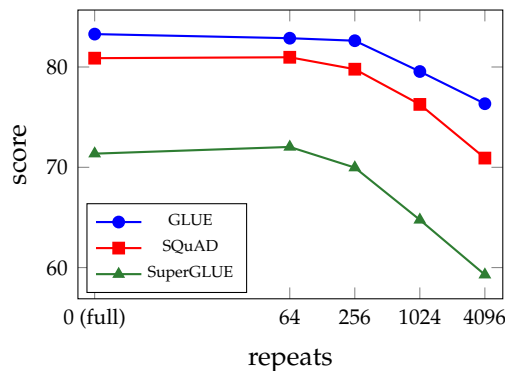
Number of tokens	Repeats	GLUE	CNN4	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Full dataset	0	83.28	19.24	80.88	71.36	26.98	39.82	27.65
2^{29}	64	82.87	19.19	80.97	72.03	26.83	39.74	27.63
2^{27}	256	82.62	19.20	79.78	69.97	27.02	39.71	27.33
2^{25}	1,024	79.55	18.57	76.27	64.76	26.38	39.56	26.80
2^{23}	4,096	76.34	18.33	70.92	59.29	26.37	38.84	25.81

Derivation : where the “Repeats” column comes from

Total tokens seen = 2^{35} (fixed). If the unique dataset has 2^k tokens, it is seen

$$\frac{2^{35}}{2^k} = 2^{35-k} \text{ times :} \quad \begin{array}{ll} k = 29 \rightarrow 2^6 = 64, & k = 27 \rightarrow 2^8 = 256, \\ k = 25 \rightarrow 2^{10} = 1024, & k = 23 \rightarrow 2^{12} = 4096. \end{array}$$

E.g. the slide writes $2^{35} = 2^{29} \times 2^6$ and $2^{35} = 2^{27} \times 2^8$.



Right: qualitative redraw of the slide’s curve. The smaller the dataset, the lower the training loss falls, which is the signature of memorisation.

Note on the slides

The legend on the slide’s training-loss plot lists “ 2^{27} ” twice; the curves correspond to full, 2^{29} , 2^{27} , 2^{25} , 2^{23} . What matters is the ordering: the smallest dataset (2^{23}) has by far the lowest training loss.

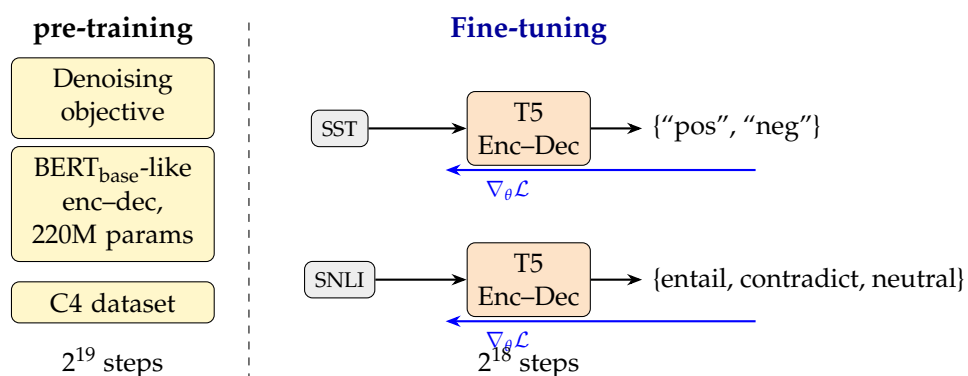
Key takeaway

- 64 repeats: only a **slight degradation** on several tasks (SQuAD and SuperGLUE even improve slightly).
- 256 repeats and beyond: performance **degrades** (as expected), and severely at 1,024 and 4,096.
- **Why?** The model may have **memorised** the pre-training data. To check, plot the training loss: smaller datasets reach much lower training loss. Pre-training a large model for many epochs on a small dataset leads to memorisation, not generalisation.
- **Takeaway:** scaling the pre-training dataset (and avoiding repetition) may scale downstream performance. Use large, clean datasets.

Module 5

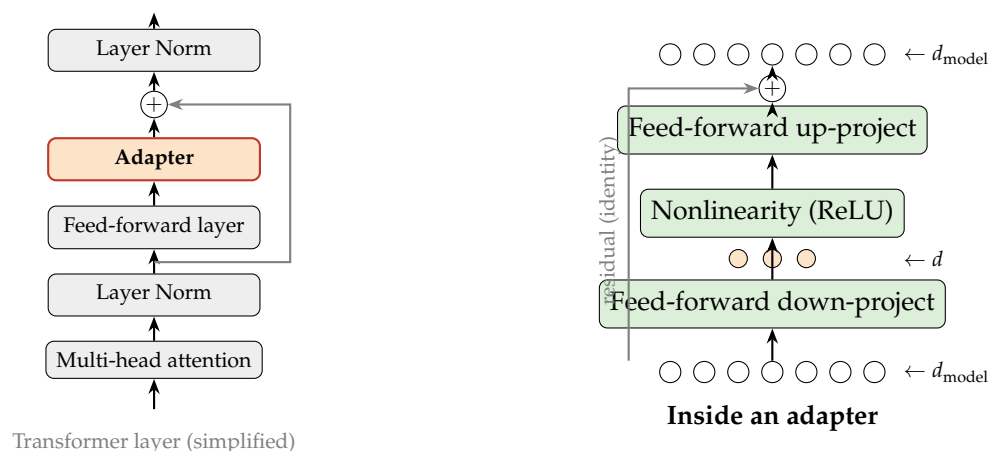
Fine-tuning strategies

5.1 The default: fine-tune all parameters



Fine-tuning *all* parameters is computationally expensive: every task needs gradients and optimizer state for all 220M parameters, plus a full copy of the model per task. Let's consider cheaper alternatives.

5.2 Adapter layers



- Add an **adapter layer** after the **FFN block** in each transformer layer.
- It is simply a **dense – ReLU – dense** block with inner dimension d and a **residual connection**.

- The number of parameters in an adapter is typically **0.5–8%** of the parameters in a transformer layer.
- During fine-tuning, **only the adapter layers and layer-normalisation parameters are updated**; everything else stays frozen.
- We can vary d and look at downstream performance.

Derivation : the adapter as an equation, with shapes

Let h $[d_{\text{model}}]$ be the output of the FFN block for one token. Then

$$\text{Adapter}(h) = h + W_{\text{up}} \text{ReLU}(W_{\text{down}}h + b_{\text{down}}) + b_{\text{up}},$$

$$W_{\text{down}} [d \times d_{\text{model}}], b_{\text{down}} [d], W_{\text{up}} [d_{\text{model}} \times d], b_{\text{up}} [d_{\text{model}}].$$

Parameter count: $2d_{\text{model}}d + d + d_{\text{model}}$.

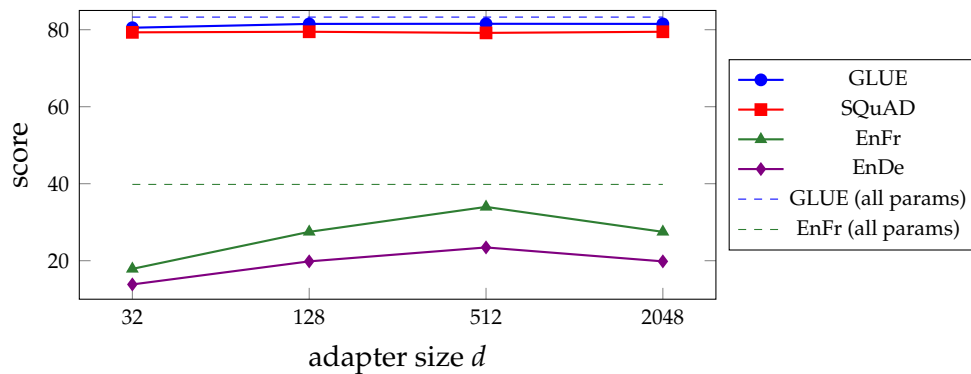
For $d_{\text{model}} = 768$, $d = 64$: $2(768)(64) + 64 + 768 = 99,136 \approx 0.1\text{M}$, versus $\approx 7.08\text{M}$ in an encoder layer ($4d_{\text{model}}^2 + 2d_{\text{model}}d_{\text{ff}}$), i.e. $\approx 1.4\%$.

Intuition : why adapters work, and why the residual is essential

- The pre-trained network already computes excellent general features; each task needs only a *small correction*. The adapter learns that correction in a low-dimensional (d) bottleneck.
- With the residual connection, if W_{up} starts near zero, the adapter is initially the **identity**, so inserting it does not damage the pre-trained model at the start of fine-tuning.
- Deployment win: store *one* frozen backbone plus a tiny adapter per task, instead of one full model per task.

5.2.1 Results

Fine-tuning method	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ All parameters	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Adapter layers, $d = 32$	80.52	15.08	79.32	60.40	13.84	17.88	15.54
Adapter layers, $d = 128$	81.51	16.62	79.47	63.03	19.83	27.50	22.63
Adapter layers, $d = 512$	81.54	17.78	79.18	64.30	23.45	33.98	25.81
Adapter layers, $d = 2048$	81.51	16.62	79.47	63.03	19.83	27.50	22.63



Key takeaway

- A small d is fine for lower-resource tasks like SQuAD ($d = 32$ gives 79.32 vs. 80.88), but worse for others.
- **Higher-resource tasks** (translation: EnFr, EnDe) **need a larger d** to perform reasonably, and even $d = 512$ is far below full fine-tuning (EnFr 33.98 vs. 39.82).

Intuition : capacity must match the amount of new information

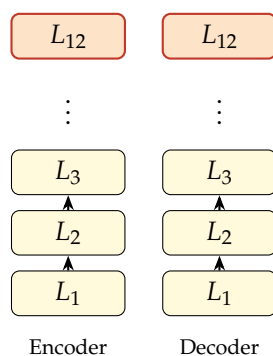
A big translation dataset carries a lot of task-specific information (a full mapping between languages) that has to be stored somewhere. A tiny adapter simply does not have the capacity. A small classification-style task needs only a small adjustment, so a small adapter is enough and also resists overfitting.

Note on the slides

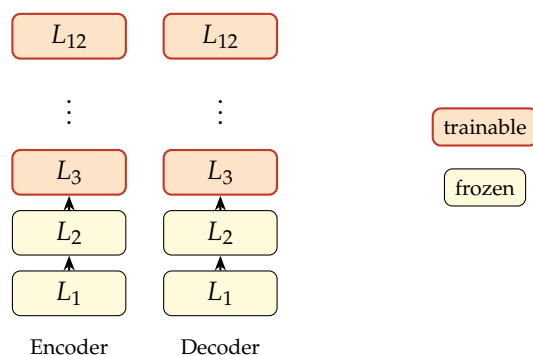
In the slide's table the $d = 2048$ row is identical to the $d = 128$ row. This is the same in the original T5 paper and looks like a copy error there, so do not read a trend into it. The reliable trend is $32 \rightarrow 128 \rightarrow 512$.

5.3 Gradual unfreezing

episode 1: top layer unfrozen



episode 10: layers 3–12 unfrozen



- **Freeze** the parameters of all layers.
- **Gradually unfreeze** layers starting from the **top** (12th) layer.
- Unfreezing happens **simultaneously for the k -th layer of the encoder and the decoder**.
- Fine-tuning steps = 2^{18} ; number of layers = 12 each. So there are **12 episodes** of unfreezing, and a new layer is unfrozen every $2^{18}/12$ steps, starting from the top.

Derivation : the schedule

Episode length = $\frac{2^{18}}{12} = \frac{262,144}{12} \approx 21,845$ steps. During episode $e \in \{1, \dots, 12\}$ the trainable layers are $L_{12}, L_{11}, \dots, L_{13-e}$ (in both stacks). The number of trainable layers grows linearly; in the final episode the whole network trains.

Why top-first? Lower layers encode general features (morphology, syntax) that transfer

across tasks, while upper layers are the most task-specific. Unfreezing from the top adapts the task-specific parts first and changes general features last and least, which reduces *catastrophic forgetting* (this idea comes from ULMFiT).

Fine-tuning method	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ All parameters	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Adapter layers, $d = 32$	80.52	15.08	79.32	60.40	13.84	17.88	15.54
Adapter layers, $d = 128$	81.51	16.62	79.47	63.03	19.83	27.50	22.63
Adapter layers, $d = 512$	81.54	17.78	79.18	64.30	23.45	33.98	25.81
Adapter layers, $d = 2048$	81.51	16.62	79.47	63.03	19.83	27.50	22.63
Gradual unfreezing	82.50	18.95	79.17	70.79	26.71	39.02	26.93

Key takeaway

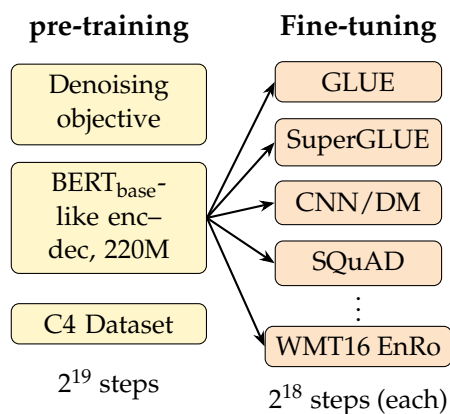
- Gradual unfreezing causes a **slight degradation** in performance.
- It **speeds up training** (early episodes need gradients for only a few layers).
- Better results may come from a more carefully tuned **unfreezing schedule**.
- Conclusion of Module 5: **fine-tuning all parameters is still best**; adapters and gradual unfreezing trade a little accuracy for efficiency.

Beyond the slides : where this leads

Parameter-efficient fine-tuning (PEFT) became a whole field after T5: LoRA (low-rank updates $\Delta W = BA$ added to frozen weights, the same bottleneck idea as adapters), prefix/prompt tuning, and QLoRA. The adapter trade-off seen here, where capacity must match task complexity, still holds for all of them.

Multi-task learning

6.1 Motivation

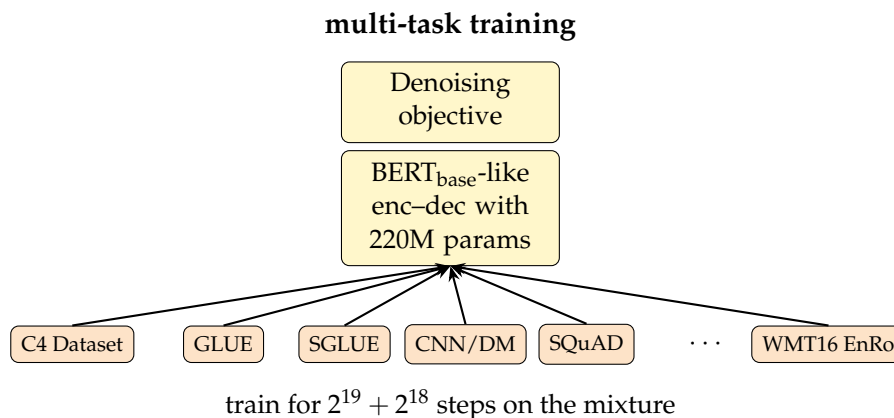


- So far we fine-tuned the pre-trained model **independently on each downstream task**.
- We observed that **adding more quality data helps** downstream performance.
- **Why not use the datasets of all supervised tasks during pre-training too?**
- In the T5 framework this simply means **mixing the datasets together and training jointly**, with the same maximum-likelihood loss, instead of a separate loss function per task.

Intuition : why text-to-text makes multi-task learning trivial

In BERT-style multi-task learning, each task needs its own head and loss, and you must balance them. In T5, every example, whether a C4 span-corruption example or “translate English to German: ...”, is just an (input text, target text) pair. You can literally concatenate the datasets and shuffle. The task prefix tells the model which task each example belongs to.

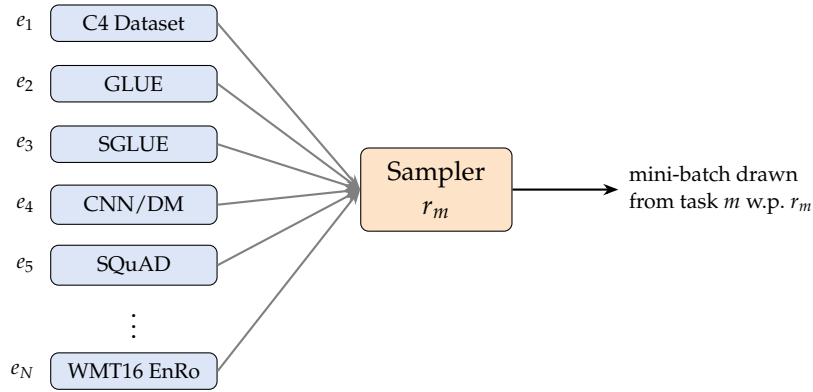
6.2 General multi-task learning



Pre-training (unsupervised C4) and all supervised tasks are **mixed into one training stream**. The total steps equal pre-training plus fine-tuning ($2^{19} + 2^{18}$), so compute matches the baseline.

6.3 How to mix: sampling strategies

Dataset sizes are very non-uniform (C4 is effectively infinite; RTE has 2.5k examples). A naive concatenation would make the model see almost only C4. We need a sampling strategy that does not favour one task at the expense of others.



6.3.1 Examples-proportional mixing

Let there be N tasks with e_n examples in task n ($n \in \{1, \dots, N\}$). Sampling proportionally to e_n would let the biggest dataset (C4) dominate, so we introduce an **artificial dataset-size limit** K :

$$r_m = \frac{\min(e_m, K)}{\sum_{n=1}^N \min(e_n, K)} \quad (6.1)$$

r_m is the probability that a training example comes from task m .

Derivation : properties of the formula

- $r_m \geq 0$ and $\sum_m r_m = 1$, so it is a valid probability distribution.
- **Small tasks** ($e_m < K$) keep their natural share: $r_m \propto e_m$.
- **Large tasks** ($e_m \geq K$) are *capped*: they all get the same weight K , however large they are. This is what stops C4 from swamping everything.
- $K \rightarrow \infty$: pure examples-proportional sampling (C4 dominates). $K \rightarrow 0$ (below the smallest e_n): all tasks capped equally, i.e. uniform sampling ("Equal" mixing).

6.3.2 Temperature-scaled mixing

Take the examples-proportional rates and sharpen or flatten them with a temperature T , as in a softmax:

$$\tilde{r}_m = \frac{r_m^{1/T}}{\sum_n r_n^{1/T}} \quad (6.2)$$

- $T = 1$: unchanged (examples-proportional).
- **Higher** T : the distribution becomes **more uniform** (at $T \rightarrow \infty$, $r^{1/T} \rightarrow 1$ for every task).

- **Lower T (below 1):** the distribution becomes **more skewed** toward the largest task.

Derivation : a worked example

Two tasks with $r = (0.9, 0.1)$.

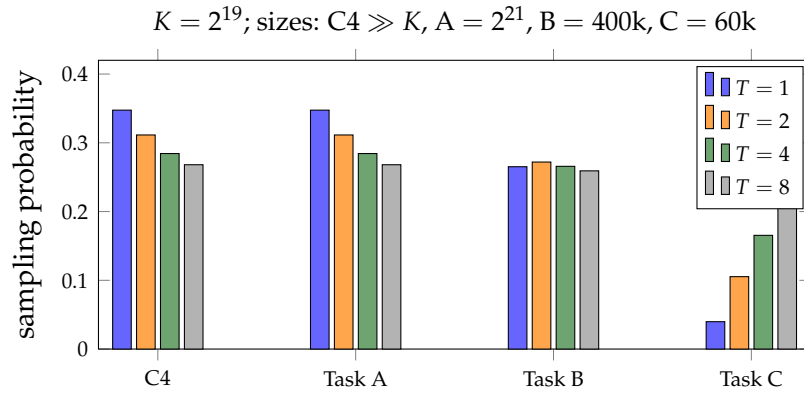
$$T = 2 : \tilde{r} \propto (\sqrt{0.9}, \sqrt{0.1}) = (0.949, 0.316) \Rightarrow \tilde{r} = (0.75, 0.25).$$

$$T = 4 : \tilde{r} \propto (0.9^{1/4}, 0.1^{1/4}) = (0.974, 0.562) \Rightarrow \tilde{r} = (0.634, 0.366).$$

The ratio between tasks goes from 9 : 1 to 3 : 1 ($= 9^{1/2}$) to $\approx 1.73 : 1$ ($= 9^{1/4}$). In general

$$\frac{\tilde{r}_a}{\tilde{r}_b} = \left(\frac{r_a}{r_b} \right)^{1/T},$$

so temperature takes the T -th root of every ratio.

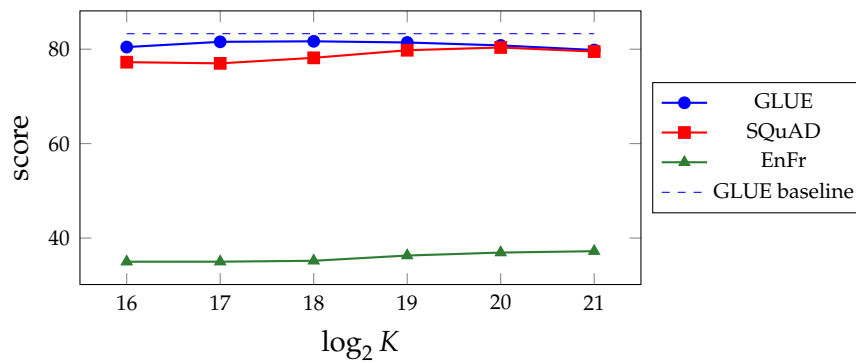


Worked numbers (see Part B for the calculation). As T grows, the smallest task (C) gets a larger share.

6.3.3 Results

Mixing strategy	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Baseline (pre-train/fine-tune)	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Equal	76.13	19.02	76.51	63.37	23.89	34.31	26.78
Examples-proportional, $K = 2^{16}$	80.45	19.04	77.25	69.95	24.35	34.99	27.10
Examples-proportional, $K = 2^{17}$	81.56	19.12	77.00	67.91	24.36	35.00	27.25
Examples-proportional, $K = 2^{18}$	81.67	19.07	78.17	67.94	24.57	35.19	27.39
Examples-proportional, $K = 2^{19}$	81.42	19.24	79.78	67.30	25.21	36.30	27.76
Examples-proportional, $K = 2^{20}$	80.80	19.24	80.36	67.38	25.66	36.93	27.68
Examples-proportional, $K = 2^{21}$	79.83	18.79	79.50	65.10	25.82	37.22	27.13
Temperature-scaled, $T = 2$	81.90	19.28	79.42	69.92	25.42	36.72	27.20
Temperature-scaled, $T = 4$	80.56	19.22	77.99	69.54	25.04	35.82	27.45
Temperature-scaled, $T = 8$	77.21	19.10	77.14	66.07	24.55	35.35	27.17

(Temperature-scaled rows use $K = 2^{21}$ before applying T .)



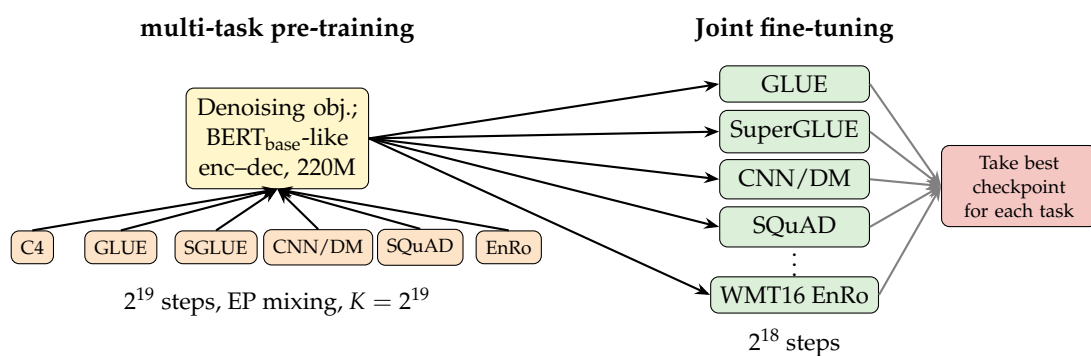
Key takeaway

- Neither increasing nor decreasing K helps much across the board; there is a **sweet spot** for K that suits some tasks (GLUE peaks near 2^{18} , SQuAD near 2^{20} , EnFr keeps improving with K).
- Temperature $T = 2$ works well for some tasks.
- **Equal mixing is the worst**: tiny tasks get sampled so often that they overfit, and big tasks (and C4) are under-trained.
- In general, **multi-task training degrades performance compared with pre-train-then-fine-tune**.

Intuition : why a single multi-task run struggles

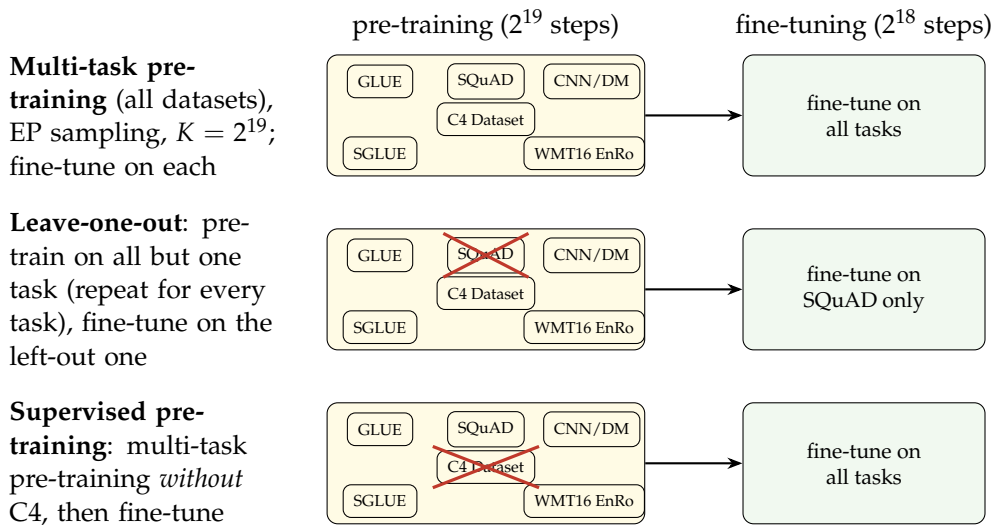
One set of weights must serve all tasks at the same point in training. Low-resource tasks overfit while high-resource ones (translation) are still under-trained; there is no single mixing rate that is right for everyone. Translation benefits from a large K (more translation data seen), while GLUE prefers a moderate K . Fine-tuning afterwards lets each task specialise.

6.4 Multi-task learning followed by fine-tuning



Pre-train on the multi-task mixture (examples-proportional, $K = 2^{19}$), then fine-tune on the supervised tasks and report the best checkpoint per task (“relaxed” multi-task learning with fine-tuning).

6.5 Three variants



- **Multi-task pre-training + fine-tuning:** use all datasets in pre-training (EP, $K = 2^{19}$), then fine-tune.
- **Leave-one-out:** pre-train on everything *except* the target task, then fine-tune on the target task; repeat for every task. This **simulates the real world**, where a pre-trained model is fine-tuned on a task it has **not seen** during pre-training.
- **Supervised multi-task pre-training:** pre-train on the supervised tasks only (no C4), then fine-tune.

6.5.1 Results

Training strategy	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Unsupervised pre-training + fine-tuning	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Multi-task training	81.42	19.24	79.78	67.30	25.21	36.30	27.76
Multi-task pre-training + fine-tuning	83.11	19.12	80.26	71.03	27.08	39.80	28.07
Leave-one-out multi-task training	81.98	19.05	79.97	71.68	26.93	39.79	27.87
Supervised multi-task pre-training	79.93	18.96	77.38	65.36	26.81	40.13	28.04

Key takeaway

- **Fine-tuning after multi-task pre-training gives performance comparable to the baseline** (GLUE 83.11 vs. 83.28; better on EnDe and EnRo).
- **Leave-one-out is only slightly worse**, which suggests that a model trained on a variety of tasks can still **adapt to new tasks**.
- **Supervised multi-task pre-training helps only on translation** (EnFr 40.13, the best). English-only unsupervised pre-training does not help translation much anyway, so removing C4 costs little there, but it hurts everything else badly.
- **Unsupervised pre-training + fine-tuning still dominates**, because it is trained on a huge amount of unlabelled data.

Note on the slides

The slide says the unsupervised model “was trained on 2^{34} tokens”; elsewhere the pre-training budget is 2^{35} tokens ($\approx 34\text{B}$). Either way, the point is that this is far more data than all the supervised datasets combined.

Beyond the slides : why multi-task pre-training is attractive anyway

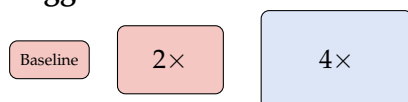
Although it only *matches* the baseline here, multi-task pre-training has a practical advantage: the model monitors downstream performance throughout pre-training, and one checkpoint serves many tasks. The final T5 models use it (Module 7), and instruction-tuned models such as FLAN-T5 push the idea to hundreds or thousands of tasks.

What if you have more compute budget?

7.1 Four ways to spend $4\times$ compute

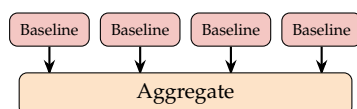
Suppose you get $4\times$ the compute. How should you use it?

1. Bigger model



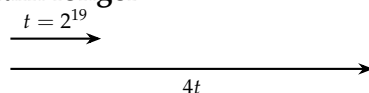
BERT_{base}-like $\rightarrow$ BERT_{large}-like: more layers, heads, larger d_{model} , d_{ff} .

2. Ensemble



4 separately pre-trained *and* fine-tuned models; average their predictions.

3. Train longer



Model sees more tokens (still a fraction of C4).

4. Bigger batch



$4\times$ more tokens in the same 2^{19} steps.

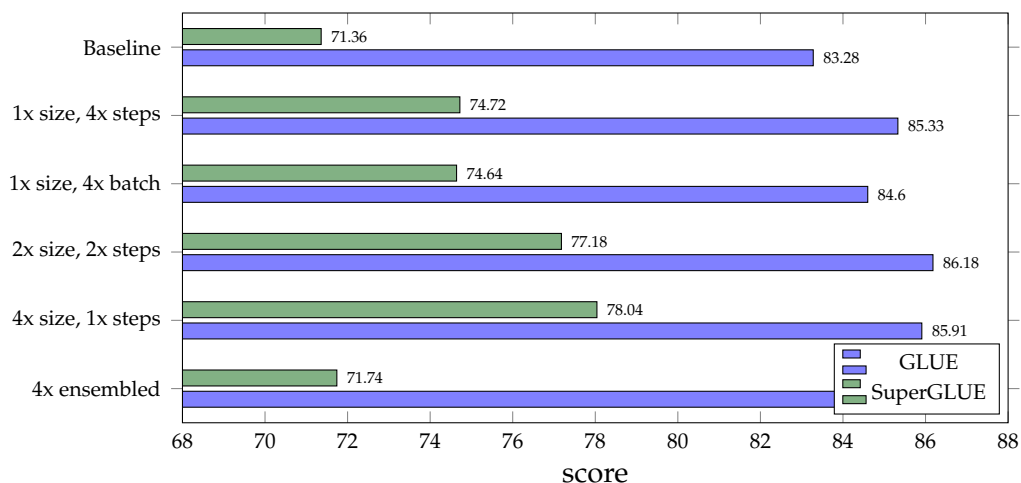
Derivation : why each option costs $\approx 4\times$ compute

Training FLOPs $\approx 6 \times P \times D$ (P = parameters, D = tokens; $2PD$ forward + $4PD$ backward).

- $4\times$ training steps: $D \rightarrow 4D \Rightarrow 4\times$.
- $4\times$ batch size: $D \rightarrow 4D \Rightarrow 4\times$.
- $2\times$ size, $2\times$ steps: $P \rightarrow 2P, D \rightarrow 2D \Rightarrow 4\times$.
- $4\times$ size, $1\times$ steps: $P \rightarrow 4P \Rightarrow 4\times$.
- $4\times$ ensemble: four baselines $\Rightarrow 4\times$ training *and* $4\times$ inference.

7.2 Results

Scaling strategy	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Baseline	83.28	19.24	80.88	71.36	26.98	39.82	27.65
1× size, 4× training steps	85.33	19.33	82.45	74.72	27.08	40.66	27.93
1× size, 4× batch size	84.60	19.42	82.52	74.64	27.07	40.60	27.84
2× size, 2× training steps	86.18	19.66	84.18	77.18	27.52	41.03	28.19
4× size, 1× training steps	85.91	19.73	83.86	78.04	27.47	40.71	28.10
4× ensembled	84.77	20.10	83.09	71.74	28.05	40.53	28.57



The lecture reads the table in steps:

1. **Training longer** (4× steps) or with a **bigger batch** (4× batch) **consistently improves the baseline**. Not a surprise: both show the model more tokens.
2. Now **increase model size 2× and limit training steps to 2t**. The model sees *fewer* tokens than in the previous case, yet **scaling the number of parameters (at the cost of fewer steps) gives better performance on many tasks** (best GLUE, SQuAD, EnFr). 4× size with 1× steps is similarly strong (best SuperGLUE).
3. **Ensembling also gives reasonable performance on many tasks, especially translation** (best CNNDM, EnDe, EnRo), but it barely helps SuperGLUE.

7.3 Parameters or training steps?

It is a **trade-off**. Scaling the number of parameters needs more compute during fine-tuning and gives a **larger inference time** (every prediction costs more). Training longer costs only at training time. Choose based on how the model will be used.

Intuition : training compute vs. lifetime compute

Training is paid once; inference is paid on every query. If a model will serve millions of requests, a smaller model trained on more tokens can be cheaper overall even if it is slightly worse. This is the reasoning behind later “compute-optimal” and “inference-optimal” train-

ing (Chinchilla, LLaMA).

7.4 Pushing the limits: the final T5 models

After the study, the authors chose:

Choice	Decision
Architecture	Encoder-decoder
Objective	Denoising with span corruption (mean span length 3, 15% corruption)
Pre-training dataset	C4
Training strategy	Multi-task pre-training (then fine-tuning)
Model size	Bigger is better
Training steps	Longer is better

Model	θ	layers	d_{model}	d_{ff}	d_{kv}	heads
Small	60M	6	512	2048	64	8
Base	220M	12	768	3072	64	12
Large	770M	24	1024	4096	64	16
3B	3B	24	1024	16384	128	32
11B	11B	24	1024	65536	128	128

(“layers” is per stack: e.g. Base has 12 encoder and 12 decoder layers.)

Derivation : inner attention width is not always d_{model}

The attention projection width is $h \cdot d_{kv}$:

$$\text{Small: } 8 \times 64 = 512 = d_{\text{model}}, \quad \text{Base: } 12 \times 64 = 768 = d_{\text{model}},$$

$$\text{Large: } 16 \times 64 = 1024 = d_{\text{model}},$$

$$\text{3B: } 32 \times 128 = 4096 = 4d_{\text{model}}, \quad \text{11B: } 128 \times 128 = 16384 = 16d_{\text{model}}.$$

For 3B and 11B, W_Q, W_K, W_V [$d_{\text{model}} \times h d_{kv}$] and W_O [$h d_{kv} \times d_{\text{model}}$] are much wider than d_{model} , and the FFN is huge ($d_{\text{ff}} = 16d_{\text{model}}$ and $64d_{\text{model}}$). The authors scaled *width* (which parallelises well on TPUs) rather than depth beyond 24 layers.

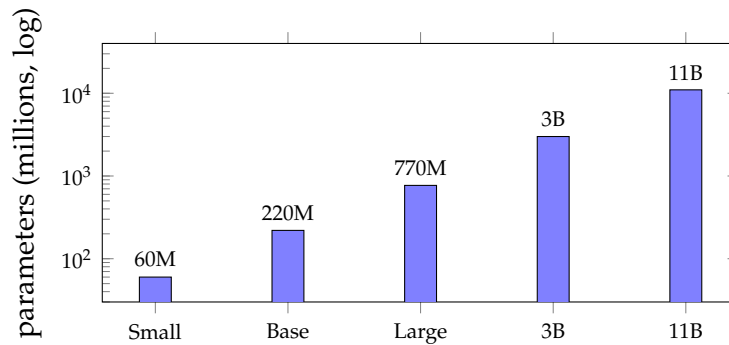
- All models are pre-trained for **1 million steps** (2^{20}) with batch size **2048** (2^{11} sequences of length 512).
- This means the model sees about **1 trillion tokens**, 32 times more than the baseline.
- We now know that more pre-training steps and bigger batches help, so with all these settings we would expect SOTA on many tasks.

Derivation : 1 trillion tokens and $32\times$

$$2^{20} \text{ steps} \times 2^{11} \frac{\text{seq}}{\text{step}} \times 2^9 \frac{\text{tokens}}{\text{seq}} = 2^{40} \approx 1.1 \times 10^{12} \text{ tokens.}$$

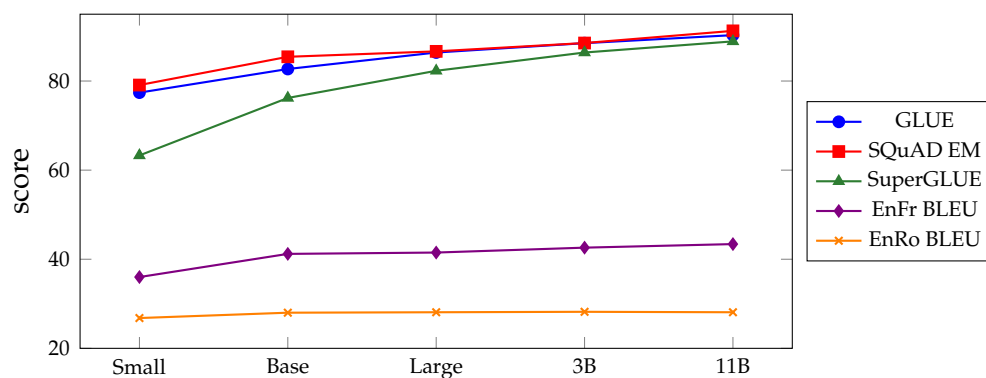
$$\frac{2^{40}}{2^{35}} = 2^5 = 32 \times \text{ the baseline.}$$

C4 has 156B tokens, so 2^{40} tokens is about 7 passes over C4. By Module 4.2's result a few repeats are harmless (64 repeats of a much smaller set only hurt slightly).



7.5 Final performance

Model	GLUE Average	CNN/DM ROUGE-2	SQuAD EM	SuperGLUE Average	WMT EnDe BLEU	WMT EnFr BLEU	WMT EnRo BLEU
Previous best	89.4	20.30	90.1	84.6	33.8	43.8	38.5
T5-Small	77.4	19.56	79.10	63.3	26.7	36.0	26.8
T5-Base	82.7	20.34	85.44	76.2	30.9	41.2	28.0
T5-Large	86.4	20.68	86.66	82.3	32.0	41.5	28.1
T5-3B	88.5	21.02	88.53	86.4	31.8	42.6	28.2
T5-11B	90.3	21.55	91.26	88.9	32.1	43.4	28.1



Key takeaway

- **Scaling the number of parameters improved performance on all tasks except translation**, which saturates (EnRo is flat from Base upwards).
- T5 reached **SOTA** on GLUE, CNN/DM, SQuAD and SuperGLUE because of this extensive study. On SuperGLUE, T5-11B (88.9) came close to the human baseline (89.8).

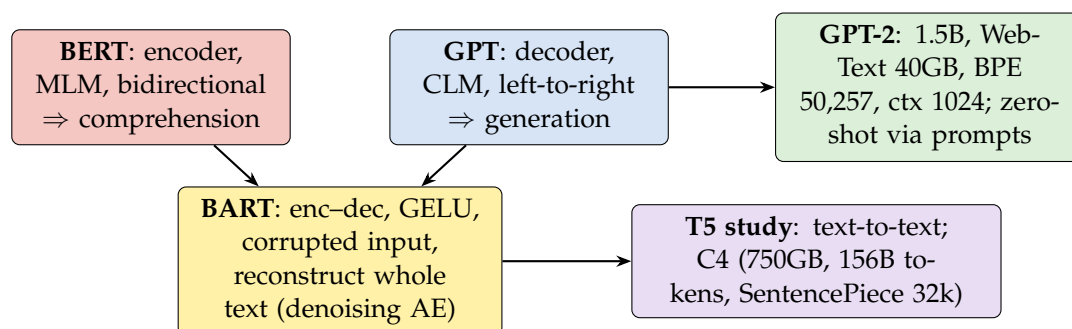
- Translation stayed below the previous best. Those systems used back-translation and large bilingual data; T5 pre-trained on English-only C4.

References (from the slides)

- Lewis et al., *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*.
- Radford et al., *Language Models are Unsupervised Multitask Learners* (GPT-2).
- Raffel et al., *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*, 2019 (T5).
- Dodge et al., *Documenting Large Webtext Corpora: A Case Study on the Colossal Clean Crawled Corpus*.
- Houlisby et al., *Parameter-Efficient Transfer Learning for NLP* (adapters).
- Presentation by Colin Raffel.

Part A

One-page revision sheet



Key takeaway : what the T5 study found

- **Baseline:** 12+12 enc-dec, 220M; 2^{19} steps $\times 2^{16}$ tokens = $2^{35} \approx 34\text{B}$; AdaFactor; $\eta = 1/\sqrt{\max(n, 10^4)}$. Fine-tuning: 2^{18} steps, $\eta = 10^{-3}$, $B = 512$, $T = 128$; best checkpoint reported.
- **Architecture:** enc-dec $>$ prefix-LM $\gg$ causal LM (fixed P , M). Sharing encoder/decoder weights is nearly free; halving depth hurts.
- **Objective:** denoising $>$ LM. BERT-style $>$ prefix-LM $\gg$ deshuffling. Replacing spans with sentinels is about the best and cheapest. Corruption rate barely matters (15%); span length 3 is best.
- **Data:** quality $>$ quantity (unfiltered C4 is worse); in-domain data helps; ≥ 256 repeats hurts (memorisation).
- **Fine-tuning:** all parameters is best. Adapters need d to grow with task size. Gradual unfreezing is slightly worse but faster.
- **Multi-task:** EP mixing $r_m = \frac{\min(e_m, K)}{\sum \min(e_n, K)}$, temperature $r^{1/T}$. Multi-task training alone is worse; multi-task pre-training + fine-tuning $\approx$ baseline; leave-one-out is slightly worse; supervised-only helps only translation.
- **Scaling:** bigger models beat longer training at equal compute ($2\times$ size, $2\times$ steps best overall); ensembles are good on translation. Final T5: 2^{20} steps $\times 2^{11} \times 512 \approx 1\text{T}$ tokens, up to 11B parameters, SOTA except translation.

A.1 Formula sheet

Concept	Formula
CLM loss	$-\sum_i \log P(x_i x_{<i})$
MLM loss	$-\sum_{i \in \mathcal{M}} \log P(x_i \tilde{x})$
BART / seq2seq loss	$-\sum_i \log P(x_i x_{<i}, \tilde{x}), \quad \text{or} \quad -\sum_j \log P(y_j y_{<j}, x)$
Prompting (GPT-2)	$p(\text{output} \text{input}) \rightarrow p(\text{output} \text{input}, \text{task})$
Perplexity	$\exp(-\frac{1}{N} \sum \log P(x_i x_{<i})); \quad \text{BPC} = -\frac{1}{N} \sum \log_2 P(c_j c_{<j})$
Decoder-only params	$\approx 12Ld_{\text{model}}^2 + Vd_{\text{model}}$ (with $d_{\text{ff}} = 4d_{\text{model}}$)
Enc-dec params	enc layer $4d_{\text{model}}^2 + 2d_{\text{model}}d_{\text{ff}}$, dec layer $8d_{\text{model}}^2 + 2d_{\text{model}}d_{\text{ff}}$
FLOPs (forward)	$\approx 2 \cdot P \cdot \text{tokens}$; training $\approx 6PD$
Tokens seen	$\text{steps} \times B \times T$ (baseline $2^{19} \cdot 2^7 \cdot 2^9 = 2^{35}$)
LR schedule	$\eta(n) = 1 / \sqrt{\max(n, k)}, k = 10^4$
Span corruption lengths	spans $s = rn / \ell$; input $n - rn + s$; target $rn + s + 1$
Prefix mask	allowed iff $j \leq \max(i, p)$; #allowed $= p^2 + \frac{n(n+1)}{2} - \frac{p(p+1)}{2}$
Adapter	$h + W_{\text{up}} \text{ReLU}(W_{\text{down}}h)$; params $2d_{\text{model}}d + d + d_{\text{model}}$
Gradual unfreezing	one more layer every $2^{18}/12$ steps, top-down
EP mixing	$r_m = \min(e_m, K) / \sum_n \min(e_n, K)$
Temperature mixing	$\tilde{r}_m = r_m^{1/T} / \sum_n r_n^{1/T}$; ratios become $(r_a/r_b)^{1/T}$
Repeats	$2^{35}/2^k$
AdaFactor memory	$n + m$ per $n \times m$ matrix (vs. $2nm$ for Adam)

A.2 Numbers worth memorising

GPT-2 sizes	117M/345M/762M/1542M	GPT-2 layers	12/24/36/48
GPT-2 d_{model}	768/1024/1280/1600	WebText	40GB, $\approx 8\text{M}$ pages, karma ≥ 3
GPT-2 vocab	50,257 (byte-level BPE)	GPT-2 ctx, batch	1024, 512
C4	750GB, 300M docs, 156B tokens	T5 vocab	32,000 (SentencePiece)
Baseline	220M, $2^{35} \approx 34\text{B}$ tokens	Fine-tune	2^{18} steps $\approx 17\text{B}$ tokens
BERT / RoBERTa tokens	137B / 2.2T	Final T5	2^{20} steps, $B = 2^{11}$, $\approx 1\text{T}$ tokens
T5 sizes	60M/220M/770M/3B/11B	Adapter share	0.5–8% of a layer
Baseline GLUE / SQuAD	83.28 / 80.88	No pre-train GLUE / SQuAD	66.22 / 50.31

Part B

Numerical problems with worked solutions

Try each problem before reading the solution. The problems follow the order of the modules.

Problem 1 (Module 3: token budget)

The T5 baseline is pre-trained with context length $T = 512$, batch size $B = 128$ and 2^{19} steps. (a) How many tokens per batch? (b) How many tokens in total, exactly and approximately? (c) What fraction of C4 (156B tokens) is this, and how many epochs?

Solution

- (a) $B \times T = 128 \times 512 = 2^7 \cdot 2^9 = 2^{16} = 65,536$ tokens.
- (b) $2^{19} \times 2^{16} = 2^{35} = 34,359,738,368 \approx 34.4\text{B}$ tokens.
- (c) $34.36/156 \approx 0.22$, so about 22% of C4 and 0.22 epochs. No example is repeated.

Problem 2 (Module 3: fine-tuning budget)

Fine-tuning uses $B = 512$, $T = 128$ and 2^{18} steps. (a) Tokens per batch? (b) Total fine-tuning tokens? (c) Ratio to pre-training tokens?

Solution

- (a) $512 \times 128 = 2^9 \cdot 2^7 = 2^{16}$, the same as pre-training (by design).
- (b) $2^{18} \cdot 2^{16} = 2^{34} \approx 17.2\text{B}$ tokens.
- (c) $2^{34}/2^{35} = 1/2$.

Problem 3 (Module 7: final T5 budget)

The final T5 models train for 2^{20} steps with batch 2^{11} sequences of length 512. Compute the total tokens, the ratio to the baseline, and the number of passes over C4.

Solution

- Tokens $= 2^{20} \cdot 2^{11} \cdot 2^9 = 2^{40} = 1.0995 \times 10^{12} \approx 1.1\text{T}$.
- Ratio $= 2^{40}/2^{35} = 2^5 = 32$.
- Passes over C4 $= 1.0995 \times 10^{12}/1.56 \times 10^{11} \approx 7.05$.

Problem 4 (Module 3: inverse square-root schedule)

With $\eta(n) = 1/\sqrt{\max(n, k)}$ and $k = 10^4$, compute η at $n = 1, 5000, 10^4, 4 \times 10^4, 2^{18}, 2^{19}$. By what factor does η fall between 2^{18} and 2^{19} ?

Solution

For $n \leq 10^4$: $\eta = 1/\sqrt{10^4} = 1/100 = 0.01$ (so $n = 1, 5000, 10^4$ all give 0.01).
 $n = 4 \times 10^4$: $1/\sqrt{4 \times 10^4} = 1/200 = 0.005$.
 $n = 2^{18}$: $1/\sqrt{2^{18}} = 1/2^9 = 1/512 \approx 0.001953$.
 $n = 2^{19}$: $1/\sqrt{2^{19}} = 1/(512\sqrt{2}) \approx 0.001381$.
Ratio = $\sqrt{2^{19}/2^{18}} = \sqrt{2} \approx 1.414$.

Problem 5 (Module 3: span-corruption sequence lengths)

For $n = 512$ tokens compute the encoder input length and decoder target length for (a) $r = 0.15, \ell = 3$; (b) $r = 0.5, \ell = 3$; (c) $r = 0.15, \ell = 10$. Use $s = rn/\ell$ spans (round to the nearest integer).

Solution

Formulas: input = $n - rn + s$, target = $rn + s + 1$.
(a) $rn = 76.8 \approx 77, s = 25.6 \approx 26$: input = $512 - 77 + 26 = 461$; target = $77 + 26 + 1 = 104$.
(b) $rn = 256, s = 85.3 \approx 85$: input = $512 - 256 + 85 = 341$; target = $256 + 85 + 1 = 342$.
(c) $rn \approx 77, s = 7.68 \approx 8$: input = $512 - 77 + 8 = 443$; target = $77 + 8 + 1 = 86$.
Higher corruption rates lengthen the target, so they cost more decoder compute. Longer spans shorten both sequences.

Problem 6 (Module 4.1: build an (input, target) pair)

Tokens: "Thank you for inviting me to your party last week ." (11 tokens). Corrupt the spans "for inviting me" and "your party last". Write the input and target and verify their lengths with the formulas.

Solution

Input: Thank you <X> to <Y> week . (7 tokens)
Target: <X> for inviting me <Y> your party last <Z> (9 tokens)
Check: $n = 11$, corrupted = 6, $s = 2$. Input = $11 - 6 + 2 = 7 \checkmark$; target = $6 + 2 + 1 = 9 \checkmark$.

Problem 7 (Module 1: GPT-2 parameter counts)

Using $P \approx 12Ld_{\text{model}}^2 + Vd_{\text{model}} + Td_{\text{model}}$ with $V = 50257$ and $T = 1024$, estimate the parameters of all four GPT-2 models.

Solution

L	d_{model}	$12Ld_{\text{model}}^2$	$+(V+T)d_{\text{model}}$	Total
12	768	84.93M	39.38M	124.3M
24	1024	301.99M	52.51M	354.5M
36	1280	707.79M	65.64M	773.4M
48	1600	1474.56M	82.05M	1556.6M

These match the reported 117M/345M/762M/1542M closely (the smallest is known to be 124M; the paper's 117M was a miscount).

Problem 8 (Module 7: T5-Small parameter count)

T5-Small: 6 encoder + 6 decoder layers, $d_{\text{model}} = 512$, $d_{\text{ff}} = 2048$, $V = 32000$, shared embeddings. Estimate the parameter count.

Solution

Encoder layer: $4d_{\text{model}}^2 + 2d_{\text{model}}d_{\text{ff}} = 4(262,144) + 2(512)(2048) = 1,048,576 + 2,097,152 = 3,145,728$.

Decoder layer: add cross-attention $4d_{\text{model}}^2 = 1,048,576$, giving 4,194,304.

Stacks: $6(3,145,728) + 6(4,194,304) = 44,040,192$.

Embeddings: $32000 \times 512 = 16,384,000$.

Total $\approx 60,424,192 \approx \mathbf{60M}$ ✓.

Problem 9 (Module 4: parameters vs. FLOPs)

One 12-layer stack has $P = 12(4d_{\text{model}}^2 + 2d_{\text{model}}d_{\text{ff}})$ with $d_{\text{model}} = 768$, $d_{\text{ff}} = 3072$ (ignore cross-attention and embeddings). An example has $n_x = 512$ input tokens and $n_y = 114$ target tokens. Compute the forward FLOPs ($\approx 2 \times \text{params} \times \text{tokens}$) for (a) the 12+12 enc-dec, (b) a 12-layer decoder-only LM, (c) a 24-layer decoder-only LM.

Solution

$P = 12 \times 7,077,888 = 84,934,656$.

(a) $2Pn_x + 2Pn_y = 2P(626) = 1.063 \times 10^{11}$ FLOPs. Parameters $\approx 2P$.

(b) $2P(n_x + n_y) = 1.063 \times 10^{11}$, the **same** as (a). Parameters $\approx P$ (half).

(c) $2(2P)(626) = 2.127 \times 10^{11}$, **double** (a). Parameters $\approx 2P$ (same as (a)).

So you can match either parameters (c) or FLOPs (b), not both. This is why the T5 table lists P and M separately.

Problem 10 (Module 4: counting mask cells)

For a sequence of length $n = 8$ with prefix length $p = 5$, count the allowed attention cells for full, causal, and prefix masks. What fraction of the full matrix does each allow?

Solution

Full: $n^2 = 64$ (100%).

Causal: $n(n+1)/2 = 36$ (56.25%).

Prefix: $p^2 + \frac{n(n+1)}{2} - \frac{p(p+1)}{2} = 25 + 36 - 15 = 46$ (71.9%).

Check by rows: rows 1–5 each see 5 keys (25), rows 6, 7, 8 see 6, 7, 8 (21); total 46 ✓.

Problem 11 (Module 5: adapter sizes)

For $d_{\text{model}} = 768$, compute the parameters of one adapter for $d = 32, 128, 512, 2048$ and express each as a percentage of one encoder layer (7,077,888 parameters). Which values fall in the “typical 0.5–8%” range?

Solution

Params = $2d_{\text{model}}d + d + d_{\text{model}}$:

$d = 32$: 49,952 (0.71%); $d = 128$: 197,504 (2.79%);

$d = 512$: 787,712 (11.1%); $d = 2048$: 3,148,544 (44.5%).

Only $d = 32$ and $d = 128$ are in the typical range; the larger adapters were included to probe high-resource tasks.

Problem 12 (Module 5: total adapter budget)

With one adapter ($d = 64$) in each of the 24 layers of the baseline (12 encoder + 12 decoder), how many trainable adapter parameters are there, and what fraction of the 220M model is that?

Solution

Per adapter: $2(768)(64) + 64 + 768 = 99,136$. Total: $24 \times 99,136 = 2,379,264 \approx 2.38\text{M}$, i.e. $2.38/220 \approx 1.08\%$ (plus a few thousand layer-norm parameters). You train and store about 1% of the model per task.

Problem 13 (Module 5: gradual-unfreezing schedule)

With 2^{18} fine-tuning steps and 12 layers per stack: (a) episode length? (b) At step 100,000 which layers are trainable? (c) What fraction of transformer layers is trainable then?

Solution

(a) $2^{18}/12 = 262,144/12 \approx 21,845.3$ steps.

(b) $100,000/21,845.3 \approx 4.58$, so we are in episode $\lfloor 4.58 \rfloor + 1 = 5$. Trainable: $L_{12}, L_{11}, L_{10}, L_9, L_8$ in both the encoder and the decoder.

(c) $5/12 \approx 41.7\%$.

Problem 14 (Module 6: examples-proportional mixing)

Four “tasks”: C4 ($e_1 = 10^{10}$), A ($e_2 = 2^{21}$), B ($e_3 = 400,000$), C ($e_4 = 60,000$). Compute r_m for (a) $K = 2^{19}$ and (b) $K = 2^{16}$. Comment.

Solution

(a) $K = 524,288$. Capped sizes: 524,288, 524,288, 400,000, 60,000; sum = 1,508,576.

$r = (0.3475, 0.3475, 0.2652, 0.0398)$.

(b) $K = 65,536$. Capped sizes: 65,536, 65,536, 65,536, 60,000; sum = 256,608.

$r = (0.2554, 0.2554, 0.2554, 0.2338)$, almost uniform.

Comment: a small K drives mixing towards “Equal”, which the T5 table shows is harmful (tiny tasks overfit, C4 and translation are under-trained). A large K lets big datasets dominate.

Problem 15 (Module 6: temperature scaling)

(a) With $r = (0.9, 0.1)$ compute $\tilde{r}$ for $T = 2$ and $T = 4$. (b) Apply $T = 2$ to the rates from part (a) of the previous problem. (c) Prove that $\tilde{r}_a/\tilde{r}_b = (r_a/r_b)^{1/T}$.

Solution

(a) $T = 2$: $(\sqrt{0.9}, \sqrt{0.1}) = (0.9487, 0.3162)$, sum 1.2649, giving $\tilde{r} = (0.75, 0.25)$.
 $T = 4$: $(0.9740, 0.5623)$, sum 1.5363, giving $\tilde{r} = (0.634, 0.366)$.
 (b) $\sqrt{\tilde{r}} = (0.5895, 0.5895, 0.5149, 0.1994)$, sum 1.8933, giving $\tilde{r} = (0.311, 0.311, 0.272, 0.105)$.
 Task C's share rises from 4.0% to 10.5%.
 (c) $\tilde{r}_a/\tilde{r}_b = \frac{r_a^{1/T}/Z}{r_b^{1/T}/Z} = (r_a/r_b)^{1/T}$, since the normaliser Z cancels. ■

Problem 16 (Module 4.2: repeats)

(a) Complete: dataset of $2^{29}, 2^{27}, 2^{25}, 2^{23}$ tokens with total budget 2^{35} ; number of repeats? (b) Wikipedia+TBC is 20GB; at 4.8 bytes/token, how many passes would a 2^{35} -token run need?

Solution

(a) 2^{35-k} : $2^6 = 64, 2^8 = 256, 2^{10} = 1024, 2^{12} = 4096$.
 (b) Tokens $\approx 20 \times 10^9 / 4.8 \approx 4.17 \times 10^9$. Passes $= 3.436 \times 10^{10} / 4.17 \times 10^9 \approx 8.2$. This is well below the 64 repeats that caused only slight degradation, so it is safe at this budget, but not for a 1T-token run (≈ 264 passes).

Problem 17 (Module 1: perplexity and BPC)

(a) A model's average cross-entropy is 2.9 nats/token. What is its perplexity? (b) GPT-2 gets 0.98 BPC on text8. What is the per-character perplexity, and the loss in nats/char?

Solution

(a) $\text{PPL} = e^{2.9} \approx 18.17$.
 (b) Per-char perplexity $= 2^{0.98} \approx 1.97$ (about a 2-way uncertainty per character). Loss in nats $= 0.98 \ln 2 \approx 0.679$ nats/char.

Problem 18 (Module 3: AdaFactor memory)

(a) How many extra floats does Adam keep for the 220M-parameter baseline, and how many GB at 4 bytes/float? (b) For one 768×3072 FFN matrix, compare Adam with factored AdaFactor.

Solution

(a) $2 \times 220\text{M} = 440\text{M}$ floats, i.e. $440\text{M} \times 4 \text{ B} = 1.76\text{GB}$ of optimizer state (on top of the weights and gradients).
 (b) Adam: $2 \times 768 \times 3072 = 4,718,592$ floats. AdaFactor: $768 + 3072 = 3,840$ floats, about

$1,229 \times$ less.

Problem 19 (Module 7: training compute)

Using $C \approx 6PD$, estimate the training compute of (a) the baseline ($P = 220\text{M}$, $D = 2^{35}$) and (b) T5-11B ($P = 11\text{B}$, $D = 2^{40}$). Ratio?

Solution

(a) $6 \times 2.2 \times 10^8 \times 3.436 \times 10^{10} \approx 4.5 \times 10^{19}$ FLOPs.
 (b) $6 \times 1.1 \times 10^{10} \times 1.0995 \times 10^{12} \approx 7.3 \times 10^{22}$ FLOPs.
 Ratio = $(11\text{B}/220\text{M}) \times 2^5 = 50 \times 32 = 1600$.

Problem 20 (Module 3: is a difference significant?)

The baseline's standard deviations are GLUE 0.235 and SQuAD 0.343. The shared enc-dec gets GLUE 82.81 and SQuAD 80.63 (baseline 83.28, 80.88). Are the drops meaningful at a 2σ threshold?

Solution

GLUE drop = $0.47 = 2.0\sigma$: borderline, just at the threshold.
 SQuAD drop = $0.25 = 0.73\sigma$: within noise.
 This supports the lecture's reading: sharing causes only a *slight* decrease.

Problem 21 (Module 1: WebText statistics)

WebText is 40GB over about 8M pages. (a) Average page size? (b) At about 4 bytes per BPE token, average tokens per page? Compare with GPT-2's context window.

Solution

(a) $40 \times 10^9 / 8 \times 10^6 = 5,000$ bytes $\approx 5\text{KB}$ per page.
 (b) $5000/4 \approx 1250$ tokens, comparable to (slightly more than) the 1024-token context, so a typical document fits roughly in one window.

Problem 22 (Module 1: GPT-2 tokens per step)

GPT-2 uses batch size 512 and context 1024. Tokens per step? If it trained for X steps to see 40GB once (about 10B tokens at 4 bytes/token), what is X ?

Solution

Tokens/step = $512 \times 1024 = 524,288 = 2^{19}$. $X \approx 10^{10} / 5.24 \times 10^5 \approx 19,000$ steps per epoch.

Problem 23 (Module 3: decoder savings)

Compare the decoder compute for BART-style full reconstruction (target = 512) and T5 span corruption (target ≈ 104 , Problem 5a), assuming decoder FLOPs are proportional to target length.

Solution

Ratio = $104/512 \approx 0.203$, so span corruption needs about 20% of the decoder compute (about $4.9\times$ less). The encoder is also slightly cheaper (461 vs. 512 tokens).

Problem 24 (Module 4.1: expected i.i.d. span length)

If each token is corrupted independently with probability $r = 0.15$ and consecutive corrupted tokens are merged into one span, what is the expected span length?

Solution

Given a span has started, it continues with probability r at each next token, so the span length is geometric with success (stop) probability $1 - r$:

$$\mathbb{E}[\ell] = \frac{1}{1 - r} = \frac{1}{0.85} \approx 1.18.$$

So the i.i.d. baseline has very short spans (≈ 1.2). With $rn \approx 77$ corrupted tokens there are ≈ 65 spans, and the target is $\approx 77 + 65 + 1 = 143$ tokens, versus ≈ 104 with $\ell = 3$.

Part C

Subjective questions with model answers

Q1. Why is BART called a “bidirectional autoregressive” transformer? Explain how it combines the strengths of BERT and GPT.

Solution

BART is an encoder–decoder. The **encoder** reads a corrupted version $\tilde{x}$ of the text with fully visible (bidirectional) self-attention, like BERT, so every token’s representation uses left and right context. The **decoder** generates the original text left to right with causal self-attention, like GPT, and reads the encoder output through cross-attention. The loss $-\sum_i \log P(x_i | x_{<i}, \tilde{x})$ is on *every* token (as in GPT), and each prediction conditions on a bidirectional view of the whole document (as in BERT). The result understands well (bidirectional encoder) and generates naturally (autoregressive decoder), which makes it strong on both comprehension and generation tasks such as summarisation.

Q2. List the ways BART corrupts its input. Why does a denoising objective produce useful representations?

Solution

Token masking, token deletion, text infilling (a span becomes one mask), sentence permutation and document rotation. Undoing each corruption requires a different kind of knowledge: masking needs lexical and semantic context, deletion needs to know *where* something is missing, infilling needs to know *how much* is missing, permutation and rotation need discourse-level understanding of what comes first. To reconstruct the clean text the model must build representations that capture syntax, semantics and document structure, which are exactly the features downstream tasks need. This is the denoising-autoencoder view.

Q3. What limitations of “pre-train then fine-tune” motivated GPT-2’s prompting approach?

Solution

(1) Each task needs its own copy of the model and its own fine-tuning run, which is expensive for large models. (2) Fine-tuning still needs thousands of labelled examples, which many tasks do not have. (3) It is unlike how humans work: once we know a language, we can do a new task from an instruction alone. GPT-2 instead conditions a single LM on a natural-language task description, modelling $p(\text{output} | \text{input}, \text{task})$ instead of $p(\text{output} | \text{input})$. Because the task is

expressed in words, no new parameters or gradient updates are needed: zero-shot transfer.

Q4. Why can a language model trained only on next-token prediction perform translation or summarisation zero-shot? Why was WebText important?

Solution

Web text contains *naturally occurring demonstrations* of many tasks: bilingual phrases (“as they say in French: ...”), summaries after “TL;DR:”, Q&A formats, and more. To minimise next-token loss on such text, the model must implicitly learn these mappings. A prompt with the right format puts the model in the context where the task’s answer is the most likely continuation. WebText mattered because it was *diverse* (many domains and formats, so many implicit tasks) and *higher quality* (Reddit karma ≥ 3 as a human filter), while excluding Wikipedia to avoid overlap with evaluation data.

Q5. GPT-2 was said to “underfit” WebText. What evidence supported this, and why was it considered encouraging?

Solution

Training and held-out perplexities on WebText were almost equal and both kept decreasing as the model grew from 117M to 1.5B parameters. If the model were overfitting, test perplexity would diverge from train perplexity. Underfitting means extra capacity would still be used productively, so larger models (and more data) should improve further, including zero-shot task performance. This trend motivated GPT-3.

Q6. Why is it hard to conclude from the GLUE table why BERT beats GPT? How does T5 address this?

Solution

GPT and BERT differ at the same time in pre-training data (books vs. books + Wikipedia), objective (CLM vs. MLM), size (117M vs. up to 336M), training duration, and fine-tuning recipe (new linear layer vs. [CLS]-based fine-tuning). With many simultaneous differences, the effect of any one is confounded. T5 runs a controlled study: fix a baseline and change *one factor at a time* (architecture, objective, corruption strategy, data, fine-tuning method, multi-task mixing, scaling), evaluating on the same diverse benchmark suite. The text-to-text format makes this possible, since every task and variant shares one model interface and one loss.

Q7. Explain the text-to-text framework. How are classification and regression cast as text generation?

Solution

Every task is “input text $\rightarrow$ output text”, with a task prefix telling the model what to do (“translate English to German:”, “summarize:”, “cola sentence:”). Classification: the target is the label word, e.g. “entailment” or “positive”. Regression (STS-B): scores in $[0, 5]$ are rounded to the

nearest 0.2 and emitted as a string such as “3.8”, turning regression into 21-way classification in text. Benefits: one architecture, one loss (maximum likelihood with teacher forcing), one decoding procedure; the pre-training and fine-tuning objectives match; multi-task mixing is trivial; comparisons across variants are fair.

Q8. Describe T5’s span-corruption objective and give two reasons it was preferred.

Solution

Randomly select spans of tokens (15% of tokens in total, mean span length 3). Replace each span in the input with a unique sentinel ($\langle X \rangle$, $\langle Y \rangle$, ...). The target is the sequence of sentinels, each followed by the tokens it replaced, ending with a final sentinel. Reasons: (1) **performance**: “replace corrupted spans” was among the best corruption strategies, beating BERT-style and MASS-style on SuperGLUE, and span length 3 was best on non-translation tasks; (2) **efficiency**: targets are short (only corrupted content) and inputs shrink (one sentinel per span), which cuts training cost relative to reconstructing the full sequence.

Q9. Explain the parameters-versus-FLOPs dilemma when comparing an encoder–decoder with a decoder-only LM.

Solution

An $L + L$ encoder–decoder has about the same number of parameters as a $2L$ -layer LM. However, its encoder parameters process only the input tokens and its decoder parameters only the target tokens, so its compute per example equals that of an L -layer LM processing input + target: $2P(n_x + n_y)$. The $2L$ LM would cost twice that. So an LM can match the enc–dec in parameters *or* in FLOPs, but not both. T5 therefore reports both P and M for every variant and compares like with like (e.g. the shared-parameter enc–dec has P params and M FLOPs, the same as a L -layer LM).

Q10. What is a prefix LM? How does its mask differ from a causal mask, and why does it perform close to an encoder–decoder?

Solution

A prefix LM is a decoder-only transformer whose self-attention mask is fully visible over a prefix (the input or task description) and causal over the rest (the output). In mask terms, query i may attend to key j iff $j \leq \max(i, p)$. It keeps generation autoregressive but gives the input bidirectional context, which a causal LM lacks. That is the main advantage of the encoder in an enc–dec, so with the same P and M the prefix LM performs close to the enc–dec (GLUE 81.82 vs. 82.81 for the shared enc–dec) and far better than a causal LM (74.70).

Q11. Summarise the effect of the unsupervised objective on downstream performance.**Solution**

(1) Denoising objectives beat language-modelling objectives for every architecture. (2) Among high-level objectives, BERT-style (reconstruct from corrupted input) is best overall; prefix LM is competitive on translation; deshuffling is clearly worst. (3) Among BERT-style corruption strategies (MASS-style masking, replacing spans, dropping tokens) all perform similarly; replacing spans with sentinels is chosen because it performs well and gives short targets. Dropping tokens helps GLUE (thanks to CoLA) but hurts SuperGLUE. (4) The corruption rate (10–50%) has minimal effect; 15% is kept. (5) Span length matters a little; 3 is best on non-translation tasks and 10 underperforms.

Q12. “More data is always better.” Critically evaluate this in light of the C4 experiments.**Solution**

Partly false. *Unfiltered* C4 is $8\times$ larger than C4 but performed worse on every task: quality beats quantity. Small, high-quality or in-domain datasets (RealNews-like, WebText-like, Wikipedia+TBC; $20\text{--}40\times$ smaller) matched or beat C4 on several tasks, and in-domain data helped matching benchmarks (Wikipedia+TBC on SQuAD and SuperGLUE). However, small datasets must be repeated to fill a fixed token budget. Up to 64 repeats costs little, but 256+ repeats degrades performance and the training loss drops sharply, a sign of memorisation. The correct conclusion is that you want data that is *both* clean and large enough to avoid heavy repetition.

Q13. How can we tell that a pre-trained model is memorising rather than generalising? Use the T5 experiment.**Solution**

Compare training loss and downstream performance as the unique dataset shrinks at a fixed token budget. For 2^{23} tokens (4096 repeats) the training loss falls much lower than for the full dataset, while downstream performance drops sharply (GLUE 76.34 vs. 83.28, SQuAD 70.92 vs. 80.88). Low training loss together with worse transfer means the model fits the specific training sequences rather than general linguistic structure.

Q14. Explain adapter layers: architecture, what gets trained, and when they work well.**Solution**

An adapter is a small bottleneck MLP (dense $d_{\text{model}} \rightarrow d$, ReLU, dense $d \rightarrow d_{\text{model}}$) with a residual connection, inserted after the FFN block of every transformer layer. During fine-tuning only the adapters and layer-norm parameters are updated; the rest of the pre-trained model is frozen. Each adapter has $2d_{\text{model}}d + d + d_{\text{model}}$ parameters (typically 0.5–8% of a layer). With the residual path, an adapter starts close to the identity, so the pre-trained behaviour is preserved initially. Results: small d is enough for low-resource tasks like SQuAD, but high-resource tasks such as translation need a larger d and still trail full fine-tuning. Capacity must

scale with how much task-specific information must be learned.

Q15. What is gradual unfreezing? Why unfreeze from the top? What did T5 find?

Solution

Start with all layers frozen and, over the course of fine-tuning, progressively unfreeze layers from the top (layer 12) downward, the k -th layer of encoder and decoder at the same time. With 2^{18} steps and 12 layers, one more layer is unfrozen every $2^{18}/12$ steps. Top layers are the most task-specific and bottom layers the most general, so adapting the top first limits catastrophic forgetting of general features. T5 found a slight degradation vs. fine-tuning all parameters, but faster training; a better-tuned schedule might close the gap.

Q16. Compare examples-proportional and temperature-scaled mixing. Why is naive or equal mixing a bad idea?

Solution

Examples-proportional: $r_m \propto \min(e_m, K)$. The cap K prevents huge datasets (C4, WMT EnFr) from swamping the others while small datasets keep their natural share. Temperature scaling then raises the rates to $1/T$ and renormalises: $T > 1$ flattens toward uniform, $T < 1$ sharpens. Naive proportional mixing (no cap) means small tasks are almost never seen. Equal mixing means small tasks are seen so often that the model overfits them, while large tasks and unsupervised data are under-trained; it gave the worst results (GLUE 76.13). Even the best mixing did not beat pre-train-then-fine-tune, because one mixing rate cannot suit every task.

Q17. Discuss the three multi-task variants (multi-task pre-training + FT, leave-one-out, supervised pre-training) and what each result teaches.

Solution

Multi-task pre-training + fine-tuning roughly matches the unsupervised baseline (GLUE 83.11 vs. 83.28, better EnDe and EnRo), so mixing supervised tasks into pre-training does no harm and gives a model that is useful throughout training. **Leave-one-out** (pre-train on all tasks except the target, then fine-tune on it) is only slightly worse, which shows that multi-task pre-training transfers to *unseen* tasks, as in the real world. **Supervised-only pre-training** (no C4) is clearly worse except on translation (EnFr best at 40.13), because English-only unsupervised data helps translation little while the huge unlabelled corpus is what drives language understanding. Unsupervised pre-training on a large corpus remains the core ingredient.

Q18. Given $4\times$ compute, what are the options and what did T5 find? When would you prefer each?

Solution

Options: train $4\times$ longer, use a $4\times$ batch, a $2\times$ model for $2\times$ steps, a $4\times$ model for $1\times$ steps, or an ensemble of 4 models. All help. Longer training and bigger batches consistently improve the baseline. Increasing model size, even at the cost of seeing fewer tokens ($2\times$ size, $2\times$ steps), gives the best results on many tasks (GLUE 86.18, SQuAD 84.18); $4\times$ size gives the best SuperGLUE. Ensembles are strong on translation and summarisation but weak on SuperGLUE, and cost $4\times$ at inference. Prefer bigger models when inference cost is acceptable and quality matters most; prefer training longer (with a smaller model) when the model will serve many queries and inference cost dominates.

Q19. List T5’s final design choices and justify each with the experiment that supports it.

Solution

- **Encoder–decoder:** best in the architecture study at equal FLOPs; prefix-LM close; causal LM far behind.
- **Denoising with span corruption** (15%, mean span 3): denoising beats LM; replacing spans performs well and is cheap; span 3 best on non-translation tasks.
- **C4:** large and clean. Unfiltered data hurt; small datasets would need harmful repetition at T5 scale.
- **Multi-task pre-training + fine-tuning:** matched the baseline and lets one model be monitored on all tasks.
- **Bigger models, longer training:** scaling parameters and steps helped in Module 7; final models trained for 2^{20} steps with 2^{11} -sequence batches ($\approx 1\text{T}$ tokens), up to 11B parameters. Result: SOTA on GLUE, SuperGLUE, SQuAD and CNN/DM; translation lagged behind specialised systems.

Q20. Why did scaling T5 fail to reach SOTA on translation, even at 11B?

Solution

C4 is English-only, so pre-training gives little knowledge of German, French or Romanian beyond what the fine-tuning data provides. The previous best translation systems used back-translation and large bilingual corpora. Consistent with Module 6, translation benefits mostly from supervised parallel data, and translation BLEU in the scaling table saturates (EnRo ≈ 28 from Base to 11B). Model capacity cannot compensate for data the model never saw.

Q21. Compare BART and T5 in terms of architecture, corruption and targets.

Solution

Both are encoder–decoder transformers pre-trained with denoising. **BART** uses several noise types (masking, deletion, infilling, sentence permutation, document rotation) and its decoder reconstructs the *entire* original text, so the loss is on every token, and it uses GELU. **T5** uses

span corruption with sentinels, and its decoder outputs *only* the corrupted spans, which is cheaper. T5 also frames every downstream task as text-to-text with task prefixes and uses relative position biases (beyond the slides). The T5 study found that the corruption strategy matters less than choosing a denoising objective at all.

Part D

Multiple-choice questions with answers and explanations

Some questions have more than one correct option; these are marked [MSQ].

D.1 Module 1: BART and GPT-2

Q1. BART's decoder computes the loss over:

- (a) only the corrupted spans with sentinels
- (b) only the first token
- (c) only the masked tokens
- (d) the entire original sequence

Answer: (d). BART reconstructs the full original text, so the loss is summed over all positions, as in GPT. Option (a) describes T5.

Q2. Which activation function does BART use?

- (a) GELU
- (b) tanh
- (c) SwiGLU
- (d) ReLU

Answer: (a). The slide describes BART as “a vanilla transformer architecture with GELU activation”.

Q3. [MSQ] Which of the following are corruption schemes used by BART?

- (a) Document rotation
- (b) Token deletion
- (c) Back-translation
- (d) Random masking

Answer: (a), (b), (d). Back-translation is a data-augmentation method for machine translation, not a BART noise function. BART also uses text infilling and sentence permutation.

Q4. BART can best be described as a:

- (a) causal language model only
- (b) masked language model only
- (c) variational autoencoder
- (d) denoising autoencoder

Answer: (d). Corrupt the input, then reconstruct the clean version.

Q5. In GPT-2, the zero-shot formulation changes $p(\text{output} \mid \text{input})$ to:

- (a) $p(\text{input} \mid \text{output})$
- (b) $p(\text{task} \mid \text{input})$
- (c) $p(\text{output} \mid \text{input}, \text{task})$
- (d) $p(\text{output} \mid \text{task})$

Answer: (c). The task, written as plain words, is added to the conditioning context at inference.

Q6. WebText was built by:

- (a) following Reddit outbound links with at least 3 karma, excluding Wikipedia
- (b) collecting news articles
- (c) crawling Wikipedia and books
- (d) scraping all of Common Crawl

Answer: (a). Karma acts as a human quality filter. Wikipedia was excluded, partly to avoid overlap with evaluation sets.

Q7. GPT-2's vocabulary size and tokenizer are:

- (a) 32,000, SentencePiece
- (b) 50,000, character-level
- (c) 50,257, byte-level BPE
- (d) 30,522, WordPiece

Answer: (c). 30,522/WordPiece is BERT; 32,000/SentencePiece is T5.

Q8. The largest GPT-2 model has:

- (a) 48 layers, $d_{\text{model}} = 1600$
- (b) 12 layers, $d_{\text{model}} = 768$
- (c) 36 layers, $d_{\text{model}} = 1280$
- (d) 24 layers, $d_{\text{model}} = 1024$

Answer: (a). 1542M parameters; context window 1024; batch size 512.

Q9. Which metric is used for the character-level text8 benchmark?

- (a) Bits per character
- (b) Accuracy
- (c) Perplexity
- (d) BLEU

Answer: (a). $\text{BPC} = -\frac{1}{N} \sum \log_2 P(c_j \mid c_{<j})$. WikiText and 1BW use perplexity; LAMBADA uses accuracy.

Q10. LAMBADA tests a model's ability to:

- (a) predict the last word of a passage using broad context
- (b) answer extractive questions
- (c) classify sentiment
- (d) translate English to French

Answer: (a). About 10,000 BooksCorpus passages; the final word of the last sentence must be predicted, which needs long-range context.

Q11. Which observation led to the conclusion that GPT-2 1.5B "still underfits" WebText?

- (a) Loss diverging during training
- (b) Train perplexity much lower than test perplexity
- (c) Train and test perplexity close and both still decreasing with model size
- (d) Test perplexity increasing with size

Answer: (c). Overfitting would show a train-test gap; instead both kept improving with scale.

D.2 Module 2: choices and T5

Q12. The main purpose of the T5 study was to:

- (a) build a multilingual model
- (b) systematically compare design choices while keeping other factors fixed
- (c) introduce the GELU activation
- (d) propose a new attention mechanism

Answer: (b). A controlled ablation study over architecture, objective, data, fine-tuning, multi-task learning and scaling.

Q13. In T5, the STS-B regression task is handled by:

- (a) predicting a class index
- (b) discarding the task
- (c) adding a regression head
- (d) outputting the score as a text string, e.g. "3.8"

Answer: (d). Everything is text-to-text; scores are rounded to increments of 0.2 and emitted as strings.

Q14. C4 contains approximately:

- (a) 750GB, 300M documents, 156B tokens
- (b) 16GB, Wikipedia only
- (c) 40GB, 8M pages
- (d) 6.1TB, 1T tokens

Answer: (a). (c) is WebText; 6.1TB is unfiltered C4; 16GB is Wikipedia.

Q15. [MSQ] Which were items on the lecture's "wishlist"?

- (a) An architecture that can use both CLM- and MLM-like objectives
- (b) Removing the decoder entirely
- (c) A pre-training dataset as large as possible
- (d) Same objective for pre-training and fine-tuning

Answer: (a), (c), (d). The enc-dec text-to-text setup satisfies all three.

D.3 Module 3: baseline

Q16. The baseline T5 model has approximately how many parameters?

- (a) 110M
- (b) 220M
- (c) 340M
- (d) 770M

Answer: (b). 12 encoder + 12 decoder layers, BERT_{base}-sized each: about twice BERT_{base} (110M).

Q17. Total pre-training tokens for the baseline are:

- (a) 2^{16}
- (b) $2^{40} \approx 1\text{T}$
- (c) 2^{19}
- (d) $2^{35} \approx 34\text{B}$

Answer: (d). 2^{19} steps $\times$ 2^{16} tokens per batch (128×512).

Q18. With $\eta(n) = 1/\sqrt{\max(n, 10^4)}$, the learning rate at step 2,500 is:

- (a) 0.0004
- (b) 0.02
- (c) 0.01
- (d) 0.001

Answer: (c). $\max(2500, 10^4) = 10^4$, so $\eta = 1/100$.

Q19. Why was AdaFactor used instead of Adam?

- (a) It is required for encoder-decoders
- (b) It converges in fewer steps
- (c) It removes the need for a learning rate
- (d) It needs less memory for optimizer state

Answer: (d). It factorises the second-moment estimate into row and column statistics ($n + m$ instead of nm numbers).

Q20. In span corruption, the input “What sets this <w> is ...” has target:

- (a) the input reversed
- (b) only the tokens replaced, each preceded by its sentinel, ending with a final sentinel
- (c) a single class label
- (d) the full original sentence

Answer: (b). E.g. “<w> mission apart <x> intelligence <y> critical descent <z>”.

Q21. During fine-tuning the baseline uses:

- (a) $B = 128, T = 512, 2^{19}$ steps
- (b) $B = 2048, T = 512, 2^{20}$ steps
- (c) $B = 512, T = 128, 2^{18}$ steps, constant $\eta = 0.001$
- (d) $B = 512, T = 1024$

Answer: (c). Same tokens per batch (2^{16}) but shorter sequences.

Q22. Without pre-training (training directly on each task), which task comes closest to the pre-trained baseline?

- (a) GLUE
- (b) SQuAD
- (c) SuperGLUE
- (d) WMT English→French

Answer: (d). EnFr 39.77 vs. 39.82: the huge supervised dataset makes pre-training almost unnecessary. SQuAD drops from 80.88 to 50.31.

Q23. Why can't the baseline's 84.24 on MNLI be directly compared to BERT_{base}'s 84.4?

- (a) MNLI was changed
- (b) Different architecture, parameter count and number of training tokens
- (c) Different random seeds only
- (d) Different metrics

Answer: (b). Enc-dec with 220M params and 34B tokens vs. encoder-only with 110M params and 137B tokens.

D.4 Module 4: architectures

Q24. In a "causal with prefix" mask with prefix length p , query i can attend to key j iff:

- (a) $j \leq \max(i, p)$
- (b) $j \leq i$
- (c) $j \geq i$
- (d) $j \leq p$

Answer: (a). Prefix keys are visible to everyone; beyond the prefix it is causal.

Q25. An enc-dec with $L + L$ layers is compared with a decoder-only LM. Which is true?

- (a) Same parameters as an L LM, same FLOPs as a $2L$ LM
- (b) Same parameters and same FLOPs as a $2L$ -layer LM
- (c) Same parameters as a $2L$ LM, same FLOPs as an L -layer LM
- (d) Half the FLOPs of an L -layer LM

Answer: (c). Encoder weights process only the input and decoder weights only the target, so compute matches an L -layer LM on the concatenation.

Q26. Sharing parameters between encoder and decoder ($\theta_{\text{Enc}} = \theta_{\text{Dec}}$):

- (a) improves all tasks
- (b) causes only a slight decrease
- (c) doubles FLOPs
- (d) severely hurts performance

Answer: (b). GLUE 83.28 $\rightarrow$ 82.81 with half the parameters and the same compute.

Q27. With the denoising objective, which architecture performed worst?

- (a) Encoder-decoder
- (b) Enc-dec, 6 layers
- (c) Causal language model
- (d) Prefix LM

Answer: (c). GLUE 74.70, SQuAD 61.14: no bidirectional context over the input.

Q28. Across all architectures in the study, which objective gave better downstream results?

- (a) LM
- (b) Denoising
- (c) They were identical
- (d) It depended entirely on the architecture

Answer: (b). "Denoising objective always results in better downstream task performance compared to language modeling objective."

D.5 Module 4.1: objectives

Q29. Among high-level objectives, which was worst?

- (a) Deshuffling
- (b) BERT-style
- (c) All equal
- (d) Prefix LM

Answer: (a). Deshuffling: GLUE 73.17, SQuAD 67.61.

Q30. MASS-style corruption differs from BERT-style in that it:

- (a) does not use random replacement of tokens
- (b) uses random replacement of tokens
- (c) deletes sentences
- (d) shuffles the input

Answer: (a). MASS-style is BERT-style without the “apple”-type random-token swaps.

Q31. “Drop corrupted tokens” did best on GLUE mainly because of:

- (a) RTE
- (b) MNLI
- (c) CoLA
- (d) SST-2

Answer: (c). It was much better on CoLA, which lifted the GLUE average, but it was worse on SuperGLUE.

Q32. What was the effect of varying the corruption rate from 10% to 50%?

- (a) Large improvement with higher rates
- (b) 50% was best everywhere
- (c) Minimal effect; 50% slightly worse on some tasks
- (d) 10% was catastrophic

Answer: (c). “Corruption rate has minimal effect on performance.”

Q33. Which mean span length outperformed the others on non-translation tasks?

- (a) 10
- (b) 3
- (c) 2
- (d) 5

Answer: (b). Span length 10 underperformed on two tasks.

D.6 Module 4.2: data

Q34. Pre-training on unfiltered C4 (6.1TB) instead of C4 (745GB):

- (a) degraded performance across tasks
- (b) only helped translation
- (c) improved all tasks
- (d) had no effect

Answer: (a). Quality matters more than raw size.

Q35. Which dataset gave the best SuperGLUE score?

- (a) RealNews-like
- (b) WebText-like
- (c) C4
- (d) Wikipedia + TBC

Answer: (d). 73.24, an in-domain effect for SuperGLUE tasks built from books and encyclopedic text.

Q36. With a total budget of 2^{35} tokens, a dataset of 2^{25} tokens is repeated:

- (a) 256 times
- (b) 4,096 times
- (c) 64 times
- (d) 1,024 times

Answer: (d). $2^{35}/2^{25} = 2^{10}$.

Q37. The training-loss curves for small repeated datasets were lower. This indicates:

- (a) better generalisation
- (b) a learning-rate bug
- (c) memorisation of the pre-training data
- (d) underfitting

Answer: (c). Lower training loss combined with worse downstream results is the signature of memorisation.

D.7 Module 5: fine-tuning

Q38. During adapter fine-tuning, which parameters are updated?

- (a) Only the embedding matrix
- (b) Only the adapter layers and layer-norm parameters
- (c) Only the final softmax layer
- (d) All parameters

Answer: (b). Everything else stays frozen.

Q39. An adapter layer with $d_{\text{model}} = 512$ and $d = 64$ has how many parameters (with biases)?

- (a) 32,768
- (b) 66,112
- (c) 65,536
- (d) 131,072

Answer: (b). $2 \cdot 512 \cdot 64 + 64 + 512 = 65,536 + 576 = 66,112$.

Q40. Which tasks need larger adapters (d) to perform reasonably?

- (a) Low-resource tasks like SQuAD
- (b) None, since d has no effect
- (c) Only GLUE
- (d) High-resource tasks like WMT translation

Answer: (d). EnFr: $d = 32 \rightarrow 17.88$, $d = 512 \rightarrow 33.98$, full fine-tuning $\rightarrow 39.82$.

Q41. In gradual unfreezing with 2^{18} steps and 12 layers, a new layer is unfrozen every:

- (a) 2^{18} steps
- (b) 12 steps
- (c) $2^{18}/12$ steps
- (d) 2^{12} steps

Answer: (c). 12 episodes, top layer first, encoder and decoder layers together.

Q42. [MSQ] Gradual unfreezing (compared with full fine-tuning):

- (a) might improve with a better schedule
- (b) requires adapters
- (c) slightly degrades performance
- (d) speeds up training

Answer: (a), (c), (d). These are the slide's three observations. Adapters are a separate method.

D.8 Module 6: multi-task learning

Q43. In examples-proportional mixing, K is:

- (a) the batch size
- (b) the temperature
- (c) the number of tasks
- (d) an artificial limit on dataset size

Answer: (d). $r_m = \min(e_m, K) / \sum_n \min(e_n, K)$.

Q44. In temperature-scaled mixing, increasing T makes the task distribution:

- (a) unchanged
- (b) concentrated on the smallest task
- (c) more skewed toward large tasks
- (d) more uniform

Answer: (d). $r^{1/T} \rightarrow 1$ as $T \rightarrow \infty$.

Q45. With rates $(0.9, 0.1)$ and $T = 2$, the new rates are:

- (a) $(0.9, 0.1)$
- (b) $(0.75, 0.25)$
- (c) $(0.5, 0.5)$
- (d) $(0.81, 0.19)$

Answer: (b). $\sqrt{0.9} : \sqrt{0.1} = 3 : 1$.

Q46. Compared with pre-train-then-fine-tune, multi-task training alone:

- (a) is identical
- (b) generally degrades performance
- (c) only fails on GLUE
- (d) is always better

Answer: (b). Best multi-task GLUE was 81.90 ($T = 2$) vs. baseline 83.28.

Q47. Leave-one-out multi-task pre-training simulates:

- (a) catastrophic forgetting
- (b) fine-tuning on a task not seen during pre-training
- (c) domain adaptation to news
- (d) ensembling

Answer: (b). It was only slightly worse, so multi-task pre-trained models adapt well to new tasks.

Q48. Supervised multi-task pre-training (no C4) helped mainly on:

- (a) SQuAD
- (b) GLUE
- (c) Translation
- (d) SuperGLUE

Answer: (c). EnFr 40.13 (best). English unsupervised data helps translation little.

D.9 Module 7: scaling

Q49. Given $4\times$ compute, which strategy gave the best GLUE and SQuAD?

- (a) $4\times$ training steps
- (b) $4\times$ batch size
- (c) $2\times$ size, $2\times$ steps
- (d) $4\times$ ensemble

Answer: (c). 86.18 GLUE, 84.18 SQuAD, even though the model sees fewer tokens than with $4\times$ steps.

Q50. Ensembling was especially helpful for:

- (a) SuperGLUE
- (b) GLUE only
- (c) translation (and summarisation)
- (d) nothing

Answer: (c). Best CNNDM (20.10), EnDe (28.05), EnRo (28.57); SuperGLUE barely improved (71.74).

Q51. A downside of scaling parameters rather than training steps is:

- (a) more data is required
- (b) it cannot use GPUs
- (c) larger inference time and more compute during fine-tuning
- (d) lower accuracy

Answer: (c). Training is paid once; inference cost is paid on every query.

Q52. Final T5 models were pre-trained for:

- (a) 2^{20} steps, batch 2^{11} , sequence length 512
- (b) 2^{18} steps, batch 512
- (c) 2^{35} steps
- (d) 2^{19} steps, batch 128

Answer: (a). About $2^{40} \approx 1\text{T}$ tokens, $32\times$ the baseline.

Q53. In T5-11B, the number of heads $\times d_{kv}$ equals:

- (a) 16384
- (b) 1024
- (c) 65536
- (d) 4096

Answer: (a). $128 \times 128 = 16384 = 16d_{\text{model}}; d_{\text{ff}} = 65536$.

Q54. Scaling T5 improved performance on all tasks except:

- (a) SuperGLUE
- (b) translation
- (c) SQuAD
- (d) GLUE

Answer: (b). Translation BLEU saturated and stayed below the previous best.

D.10 Answer key

1 d	2 a	3 abd	4 d	5 c	6 a	7 c	8 a	9 a	10 a
11 c	12 b	13 d	14 a	15 acd	16 b	17 d	18 c	19 d	20 b
21 c	22 d	23 b	24 a	25 c	26 b	27 c	28 b	29 a	30 a
31 c	32 c	33 b	34 a	35 d	36 d	37 c	38 b	39 b	40 d
41 c	42 acd	43 d	44 d	45 b	46 b	47 b	48 c	49 c	50 c
51 c	52 a	53 a	54 b						